

# C# ve ASP.NET Core MVC – Tam Teknik Referans

## ORTAM

- **Birincil dil:** C# 12 (.NET 8)
- **Web çerçevesi:** ASP.NET Core MVC 8
- **View motoru:** Razor ( `.cshtml` )
- **ORM:** Entity Framework Core 8 (EF Core)
- **Veritabanı:** Microsoft SQL Server (yerel veya uzak)
- **İstemci tarafı:** JavaScript (ES6+), jQuery 3.7.1
- **CSS çerçevesi:** Bootstrap 5
- **Geliştirme ortamı:** Visual Studio / Visual Studio Code (belirtilmemiş)
- **İşletim sistemi varsayımı:** Windows (Trusted\_Connection bağlantı dizesi nedeniyle ima edilmiş, kesin değil)
- **Örnek proje:** `HastaneMVC` — hastane randevu yönetim uygulaması

## İÇİNDEKİLER

### Bölüm I — Dil ve İstemci Tarafı Kavramları

1. JavaScript Temelleri
2. jQuery ve DOM Manipülasyonu
3. Nesne Yönelimli Programlama Temelleri
4. SOLID İlkeleri

### Bölüm II — C# Dili

5. C# Söz Dizimi ve Temel Yapılar
6. C# Sınıfları ve OOP Uygulaması

### Bölüm III — Veritabanı

7. İlişkisel Veritabanı Tablo İlişkileri
8. Referans Bütünlüğü ve Cascade Davranışları

### Bölüm IV — ASP.NET Core MVC

9. Razor View Engine
0. Tag Helpers
1. Form İşleme ve Model Binding
2. AJAX ile Asenkron Veri Akışı
3. CRUD İşlem Akışları
4. Dependency Injection ve DbContext

### Bölüm V — Proje Dokümantasyonu

5. HastaneMVC — Proje Yapısı ve Özellik Anlatımları

### Ekler

- Ek A — Tag Helper hızlı referansı
- Ek B — Anahtar kelime ve terim sözlüğü

# Bölüm 1 — JavaScript Temelleri

## 1.1 Değişkenler ve Veri Tipleri

### 1.1.1 Genel Bakış

JavaScript, **dinamik tipli** bir programlama dilidir. Bir değişken tanımlanırken tipi belirtilmez; tip, atanan değere göre çalışma zamanında belirlenir. Bu esneklik kod yazımını hızlandırır, ancak hatalı tip işlemlerini derleme zamanında değil çalışma zamanında ortaya çıkarır.

JavaScript'te üç değişken tanımlama anahtar kelimesi vardır: `let`, `const` ve `var`. Bunlardan `var` eski (ES5 öncesi) yöntemdir; modern kodda `let` ve `const` kullanılır.

### 1.1.2 Söz Dizimi

```
// ornekler/01-degisken-tanimlama.js
let degisken = baslangic_degeri;    // değiştirilebilir
const sabit = baslangic_degeri;    // sabit (yeniden atanamaz)
var eski_yontem = baslangic_degeri; // eski tarz, kaçınılmalı
```

Öğeler:

- `let` — blok kapsamlı, yeniden atanabilir
- `const` — blok kapsamlı, **bir kez atanır**, yeniden atanamaz
- `var` — fonksiyon kapsamlı, hoisting davranışı vardır

### 1.1.3 Davranış

Bir `let` veya `const` değişkeni içinde tanımlandığı süslü parantez bloğu (`{ }`) bittiğinde kapsamdan düşer. `const` ile tanımlanan değişken yeni bir değer atamasıyla değiştirilemez, ancak nesne veya dizi içerikleri **mutable** kalır — atama yasaktır, içeriği değiştirme yasak değildir.

JavaScript'in **yedi temel veri tipi** şunlardır: `string`, `number`, `boolean`, `null`, `undefined`, `object` (dizi ve nesne dahil), `symbol`. Bir değişkenin çalışma zamanı tipi `typeof` operatörü ile sorgulanır.

### 1.1.4 Örnekler

#### Örnek 1.1.1 — Tip kontrolü

Amaç: Farklı tipte değişkenler tanımlayıp `typeof` ile tipleri kontrol etmek.

```
// ornekler/01-tip-kontrol.js
let isim = "Ayşe";
let yas = 22;
let aktif = true;
let skor = null;
let adres;

console.log(typeof isim);
console.log(typeof yas);
console.log(typeof aktif);
console.log(typeof skor);
console.log(typeof adres);
```

Çıktı:

```
string
number
boolean
object
undefined
```

Not: `typeof null` ifadesinin `"object"` döndürmesi JavaScript'in tarihsel bir tasarım hatasıdır. Modern dilde değişmemiştir, geri uyumluluk için korunmaktadır.

### Örnek 1.1.2 — `const` ve nesne mutasyonu

Amaç: `const` ile tanımlanan bir nesnenin içeriğinin değiştirilebildiğini, ancak yeniden atanamadığını göstermek.

```
// ornekler/01-const-mutasyon.js
const kullanıcı = { ad: "Ali", yas: 30 };

kullanıcı.yas = 31;          // izinli — içerik değişimi
console.log(kullanıcı);

kullanıcı = { ad: "Veli" };  // hata — yeniden atama
```

Çıktı:

```
{ ad: 'Ali', yas: 31 }
TypeError: Assignment to constant variable.
```

### 1.1.5 Veri Tipi Referansı

| Tip                    | Örnek değer        | <code>typeof</code> çıktısı |
|------------------------|--------------------|-----------------------------|
| <code>string</code>    | "Merhaba"          | "string"                    |
| <code>number</code>    | 42, 3.14           | "number"                    |
| <code>boolean</code>   | true, false        | "boolean"                   |
| <code>null</code>      | null               | "object"                    |
| <code>undefined</code> | atanmamış değişken | "undefined"                 |
| <code>array</code>     | [1, 2, 3]          | "object"                    |
| <code>object</code>    | { ad: "Ali" }      | "object"                    |

### 1.1.6 Mini Sözlük

- **Değişken** (variable): Bellekte bir değer için verilen ad.
- **Sabit** (constant): Bir kez atandıktan sonra değiştirilemeyen değişken.
- **Kapsam** (scope): Bir değişkenin erişilebilir olduğu kod bölgesi.
- **Dinamik tip** (dynamic type): Tipin çalışma zamanında belirlendiği dil özelliği.
- **Hoisting**: `var` ve fonksiyon tanımlamalarının kapsamın en üstüne taşınması davranışı.

## 1.2 Karar Yapıları

### 1.2.1 Genel Bakış

Karar yapıları, bir koşulun değerine göre kod akışını dallandırır. JavaScript dört temel karar yapısı sunar: `if / else if / else`, üçlü operatör (`? :`), `switch`, ve mantıksal kısa devre (`&&`, `||`).

### 1.2.2 Söz Dizimi

```
// ornekler/01-karar-syntax.js
if (kosul1) {
  // kosul1 doğruysa
} else if (kosul2) {
  // kosul2 doğruysa
} else {
  // hiçbirisi değilse
}

// Üçlü operatör
const sonuc = kosul ? "doğru" : "yanlış";

// switch
switch (deger) {
  case "a":
    // ...
    break;
  case "b":
  case "c": // birden çok değer aynı bloğu çalıştırır
    // ...
    break;
  default:
    // ...
}
```

### 1.2.3 Davranış

`switch` ifadesinde her `case` bloğunun sonunda `break` kullanılmazsa **fall-through** davranışı oluşur — sonraki `case` bloğu da çalışır. `default` bloğu hiçbir `case` eşleşmezse devreye girer ve isteğe bağlıdır.

Üçlü operatör tek bir ifade döndürür ve atama veya dönüş değerlerinde kullanışlıdır.

### 1.2.4 Örnekler

#### Örnek 1.2.1 — Not harf karşılığı

Amaç: Sayısal nota karşılık gelen harf notunu hesaplamak.

```
// ornekler/01-not-harf.js
const not = 78;
let harf;

if (not >= 90) harf = "AA";
else if (not >= 80) harf = "BB";
else if (not >= 70) harf = "CC";
else if (not >= 60) harf = "DD";
else harf = "FF";

console.log(`Not ${not} → ${harf}`);
```

Çıktı:

```
Not 78 → CC
```

#### Örnek 1.2.2 — Üçlü operatörle durum belirleme

Amaç: Geçme durumunu tek bir ifadede belirlemek.

```
// ornekler/01-ucclu-operator.js
const not = 55;
const durum = not >= 60 ? "Geçti" : "Kaldı";
console.log(durum);
```

Çıktı:

Kaldı

### Örnek 1.2.3 — switch ile haftanın günü

Amaç: Gün adına göre hafta içi veya hafta sonu olduğunu yazdırmak.

```
// ornekler/01-switch-gun.js
const gun = "Pazartesi";

switch (gun) {
  case "Cumartesi":
  case "Pazar":
    console.log("Hafta sonu");
    break;
  default:
    console.log("Hafta içi");
}
```

Çıktı:

Hafta içi

## 1.3 Döngüler

### 1.3.1 Genel Bakış

JavaScript'te döngüler tekrarlı işlemler için kullanılır. Dört yaygın döngü yapısı vardır: `for` (sayacı), `while` (koşullu, ön kontrol), `do-while` (koşullu, son kontrol), ve `forEach` / `for-of` (koleksiyon tarama).

### 1.3.2 Söz Dizimi

```
// ornekler/01-dongu-syntax.js
// for — sayacı
for (let i = 0; i < 10; i++) {
  // gövde
}

// while — koşul önce
while (kosul) {
  // gövde
}

// do-while — gövde önce, koşul sonra (en az bir kez çalışır)
do {
  // gövde
} while (kosul);

// forEach — dizi için
[1, 2, 3].forEach((eleman, indeks) => {
  // her eleman için
});
```

### 1.3.3 Davranış

`for` döngüsü üç parçaya ayrılır: başlangıç ifadesi, koşul, ve adım ifadesi. Koşul her iterasyondan önce kontrol edilir.

`break` anahtar kelimesi döngüden tamamen çıkar; `continue` mevcut iterasyonu atlar ve bir sonrakine geçer.

`forEach` metodu, fonksiyonel bir yaklaşımla çalışır. `break` ve `continue` desteklemez — döngüyü erken sonlandırmak için `for-of` veya geleneksel `for` kullanılır.

### 1.3.4 Örnekler

#### Örnek 1.3.1 — Birikimli toplam

Amaç: `for` döngüsü ile 1'den n'e kadar toplam hesaplamak.

```
// ornekler/01-toplam.js
const n = 5;
let toplam = 0;

for (let i = 1; i <= n; i++) {
  toplam += i;
  console.log(`i=${i}   toplam=${toplam}`);
}
```

Çıktı:

```
i=1   toplam=1
i=2   toplam=3
i=3   toplam=6
i=4   toplam=10
i=5   toplam=15
```

### Örnek 1.3.2 — `break` ve `continue`

Amaç: 3'ü atlayıp 7'de durarak sayıları yazdırmak.

```
// ornekler/01-break-continue.js
for (let i = 0; i < 10; i++) {
  if (i === 3) continue;
  if (i === 7) break;
  process.stdout.write(`${i} `);
}
```

Çıktı:

```
0 1 2 4 5 6
```

### Örnek 1.3.3 — `forEach` ile dizi tarama

Amaç: Bir öğrenci listesindeki her ismi büyük harflerle yazdırmak.

```
// ornekler/01-foreach.js
const ogrenciler = ["Ali", "Veli", "Ayşe"];

ogrenciler.forEach((ad, indeks) => {
  console.log(`[${indeks}] ${ad.toUpperCase()}`);
});
```

Çıktı:

```
[0] ALI
[1] VELI
[2] AYŞE
```

## 1.4 Diziler ve Dizi Metotları

### 1.4.1 Genel Bakış

JavaScript'te diziler, içeriği değiştirilebilir (mutable), boyutu dinamik olarak büyüyüp küçülebilen koleksiyonlardır. Farklı tipte öğeleri aynı dizide tutmak mümkündür, ancak iyi uygulama tek tip içerik tutmaktır.

### 1.4.2 Söz Dizimi

```
// ornekler/01-dizi-syntax.js
const dizi = [eleman1, eleman2, eleman3];

// Eriřim
dizi[0]; // ilk eleman
dizi[dizi.length - 1]; // son eleman

// Ekleme / çıkarma
dizi.push(yeni); // sona ekle
dizi.pop(); // sondan çıkar
dizi.unshift(yeni); // başa ekle
dizi.shift(); // baştan çıkar
```

### 1.4.3 API / Üye Referansı

| Metot                     | Açıklama                                         | Dönüş                              |
|---------------------------|--------------------------------------------------|------------------------------------|
| <code>push(...)</code>    | Sona bir veya daha çok eleman ekler              | Yeni uzunluk                       |
| <code>pop()</code>        | Son elemanı çıkarır ve döndürür                  | Çıkarılan eleman                   |
| <code>unshift(...)</code> | Başta eleman ekler                               | Yeni uzunluk                       |
| <code>shift()</code>      | Baştan eleman çıkarır                            | Çıkarılan eleman                   |
| <code>filter(fn)</code>   | <code>fn</code> doğru döndüren elemanları toplar | Yeni dizi                          |
| <code>map(fn)</code>      | Her elemanı <code>fn</code> 'e göre dönüřtürür   | Yeni dizi (aynı uzunlukta)         |
| <code>find(fn)</code>     | İlk eşleşen elemanı bulur                        | Eleman veya <code>undefined</code> |
| <code>includes(x)</code>  | <code>x</code> dizide var mı?                    | <code>boolean</code>               |
| <code>indexOf(x)</code>   | İlk konumu bulur                                 | Sayı (yoksa <code>-1</code> )      |
| <code>sort()</code>       | Diziyi yerinde sıralar                           | Aynı dizi (referans)               |
| <code>length</code>       | Eleman sayısı (özellik, metot deęil)             | Sayı                               |

### 1.4.4 Örnekler

#### Örnek 1.4.1 — Ekleme, çıkarma, arama

Amaç: Bir liste üzerinde temel işlemleri uygulamak.

```
// ornekler/01-dizi-temel.js
const renkler = ["kırmızı", "mavi", "yeşil"];

renkler.push("sarı");
console.log(renkler);

renkler.pop();
console.log(renkler);

console.log(renkler.includes("mavi"));
console.log(renkler.indexOf("yeşil"));
console.log(renkler.length);
```

Çıktı:

```
[ 'kırmızı', 'mavi', 'yeşil', 'sarı' ]
[ 'kırmızı', 'mavi', 'yeşil' ]
true
2
3
```

#### Örnek 1.4.2 — `filter` ve `map` zincirleme

Amaç: Geçer not alan öğrencileri filtreleyip etiketlemek.

```
// ornekler/01-dizi-zincir.js
const notlar = [85, 42, 91, 60, 73];

const gecenler = notlar
  .filter(n => n >= 60)
  .map(n => `${n} ✓`);

console.log(gecenler);
```

Çıktı:

```
[ '85 ✓', '91 ✓', '60 ✓', '73 ✓' ]
```

## 1.5 Fonksiyonlar

### 1.5.1 Genel Bakış

JavaScript'te fonksiyonlar **birinci sınıf vatandaş** (first-class citizens): değişkenlere atanabilir, parametre olarak geçirilebilir, dönüş değeri olabilirler.

İki ana söz dizimi vardır: klasik `function` bildirimi ve **arrow function** (`=>`).

### 1.5.2 Söz Dizimi

```
// ornekler/01-fonksiyon-syntax.js
// 1) Klasik fonksiyon bildirimi
function adi(parametre1, parametre2) {
  return parametre1 + parametre2;
}

// 2) Arrow function
const adi2 = (parametre1, parametre2) => parametre1 + parametre2;

// 3) Varsayılan parametre değeri
function selamla(isim, dil = "tr") {
  return dil === "tr" ? `Merhaba ${isim}` : `Hello ${isim}`;
}
```

### 1.5.3 Davranış

Arrow function'lar kendi `this` bağlamlarına sahip değildir; üst kapsamdakini kullanırlar. Olay işleyicileri ve callback'lerde bu davranış önemlidir.

Bir fonksiyondan değer döndürmek `return` ile yapılır. Arrow function tek ifade ise `return` yazılmaz; bloklu söz dizimi (`{ }`) kullanılıyorsa `return` zorunludur.

### 1.5.4 Örnekler

#### Örnek 1.5.1 — Klasik vs arrow function

Amaç: Aynı işi iki söz dizimiyle yazmak.

```
// ornekler/01-fonksiyon-stil.js
function kare(x) {
  return x * x;
}

const kare2 = x => x * x;

console.log(kare(5));
console.log(kare2(5));
```

Çıktı:

```
25
25
```

### Örnek 1.5.2 — Varsayılan parametre

Amaç: Eksik parametre durumunda varsayılan değer kullanmak.

```
// ornekler/01-varsayilan-parametre.js
function karsila(isim, dil = "tr") {
  return dil === "tr"
    ? `Merhaba ${isim}`
    : `Hello ${isim}`;
}

console.log(karsila("Ayşe"));
console.log(karsila("Ayşe", "en"));
```

Çıktı:

```
Merhaba Ayşe
Hello Ayşe
```

### 1.5.5 Bölüm Özeti

- Değişken tanımlamada modern kod `let` ve `const` kullanır; `var` eski yöntemdir.
  - JavaScript dinamik tiplidir; tip kontrolü için `typeof` kullanılır.
  - Karar yapıları: `if/else if/else`, üçlü operatör, `switch`.
  - Döngüler: `for` (sayaçlı), `while`, `do-while`, `forEach` (dizilerde).
  - Diziler dinamik boyutludur; `push`, `pop`, `filter`, `map`, `find` en yaygın metotlardır.
  - Fonksiyonlar birinci sınıf vatandaşdır; klasik ve arrow olmak üzere iki söz dizimi vardır.
  - Arrow function'lar **kendi `this` bağlamına sahip değildir** — üst kapsamdakini paylaşır.
-

# Bölüm 2 — jQuery ve DOM Manipülasyonu

## 2.1 Genel Bakış

jQuery, DOM (Document Object Model) ile etkileşimi sadeleştiren, tarayıcı uyumluluk farklılıklarını gizleyen bir JavaScript kütüphanesidir. ASP.NET Core MVC projelerinde özellikle **client-side validation** ve **AJAX** çağrıları için yaygın kullanılır.

Modern JavaScript'in gelişimiyle jQuery'nin gerekliliği azalmıştır, ancak hâlâ **kurumsal MVC projelerinin standart parçasıdır** ve `_ValidationScriptsPartial` gibi yapılar jQuery'ye bağımlıdır.

## 2.2 Söz Dizimi ve Seçiciler

### 2.2.1 Temel söz dizimi

```
// ornekler/02-jquery-syntax.js
$(seçici) // bir veya daha çok elemanı seç
$(seçici).metot() // seçilen elemanlar üzerinde metod çağır
$(document).ready(fn) // DOM hazır olunca fn'i çalıştır
```

### 2.2.2 Seçici çeşitleri

| Seçici                                 | Açıklama                                                   |
|----------------------------------------|------------------------------------------------------------|
| <code>\$("#id")</code>                 | ID'ye göre tek eleman                                      |
| <code>\$(".sınıf")</code>              | Sınıfa göre tüm elemanlar                                  |
| <code>\$("etiket")</code>              | Etiket adına göre tüm elemanlar                            |
| <code>\$("#a.btn")</code>              | Birleşik: <code>a</code> etiketi + <code>btn</code> sınıfı |
| <code>\$("#input[type='text']")</code> | Niteliğe göre filtreleme                                   |
| <code>\$("#liste li")</code>           | İç içe seçim                                               |
| <code>this</code>                      | Olay işleyicisinde tetikleyen eleman                       |

## 2.3 Olay Bağlama (Event Binding)

### 2.3.1 Genel Bakış

jQuery'de olaylar `.metotAdi(handler)` şeklinde veya genel `.on("olay_adi", handler)` ile bağlanır. Olay tetiklendiğinde handler fonksiyonu çalışır.

### 2.3.2 Sık Kullanılan Olaylar

| Metot                  | Hangi olayı dinler                                                                      |
|------------------------|-----------------------------------------------------------------------------------------|
| <code>.click()</code>  | Tıklama                                                                                 |
| <code>.change()</code> | <code>&lt;select&gt;</code> , <code>&lt;input type="checkbox"&gt;</code> vb. değişiklik |
| <code>.keyup()</code>  | Klavye tuşu bırakıldığında                                                              |
| <code>.submit()</code> | Form gönderildiğinde                                                                    |
| <code>.hover()</code>  | Fare üzerine geldiğinde ve çıktığında                                                   |
| <code>.focus()</code>  | Bir input odaklandığında                                                                |
| <code>.blur()</code>   | Odak kaybedildiğinde                                                                    |

### 2.3.3 Örnekler

#### Örnek 2.3.1 — `.click()` ile tıklama sayacı

Amaç: Bir butona tıklandıkça sayacı arttırıp ekranda göstermek.

```
<!-- ornekler/02-click-sayac.html -->
<button id="btn">Tıkla</button>
<span id="sayac">0</span>

<script>
let sayi = 0;
$("#btn").click(function () {
    sayi++;
    $("#sayac").text(sayi);
});
</script>
```

Çıktı: (yok — yan etki: `#sayac` öğesinin metni her tıklamada artar)

#### Örnek 2.3.2 — `.change()` ile seçim değişikliği

Amaç: Bir `<select>` öğesinin değeri değiştiğinde başka bir alanda göstermek.

```
<!-- ornekler/02-change.html -->
<select id="sehir">
    <option value="">-- Şehir --</option>
    <option>İstanbul</option>
    <option>Ankara</option>
    <option>İzmir</option>
</select>
<p id="sonuc"></p>

<script>
$("#sehir").change(function () {
    $("#sonuc").text("Seçilen: " + $(this).val());
});
</script>
```

Çıktı: (yok — yan etki: kullanıcı seçim yaptıkça `#sonuc` öğesinin metni güncellenir)

#### Örnek 2.3.3 — `.keyup()` ile canlı arama benzeri etki

Amaç: Bir input'a yazıldıkça karakter sayısını göstermek.

```
<!-- ornekler/02-keyup.html -->
<input id="metin" type="text" />
<p>Karakter: <span id="adet">0</span></p>

<script>
$("#metin").keyup(function () {
    $("#adet").text($(this).val().length);
});
</script>
```

Çıktı: (yok — yan etki: yazıldıkça `#adet` öğesinin metni karakter sayısına eşitlenir)

## 2.4 İçerik ve Stil Manipülasyonu

### 2.4.1 İçerik Metotları

| Metot                    | İşlevi                                                                                     |
|--------------------------|--------------------------------------------------------------------------------------------|
| <code>.text()</code>     | Salt metin okur/yazar (HTML etiketleri kaçırılır)                                          |
| <code>.html()</code>     | HTML olarak okur/yazar (etiketler işlenir)                                                 |
| <code>.val()</code>      | Form öğesinin (input, select, textarea) değerini okur/yazar                                |
| <code>.attr(ad)</code>   | HTML niteliği okur/yazar ( <code>href</code> , <code>src</code> , <code>data-*</code> vb.) |
| <code>.prop(ad)</code>   | DOM özelliği okur/yazar ( <code>checked</code> , <code>disabled</code> vb.)                |
| <code>.append(x)</code>  | Elemanın sonuna içerik ekler                                                               |
| <code>.prepend(x)</code> | Elemanın başına içerik ekler                                                               |
| <code>.empty()</code>    | Elemanın içeriğini temizler                                                                |
| <code>.remove()</code>   | Elemanı DOM'dan kaldırır                                                                   |

## 2.4.2 Stil Metotları

| Metot                                                | İşlevi                            |
|------------------------------------------------------|-----------------------------------|
| <code>.css("ozellik", "deger")</code>                | Tek CSS özelliği ayarlar          |
| <code>.css({ a: x, b: y })</code>                    | Çoklu CSS özelliği ayarlar        |
| <code>.addClass("sinif")</code>                      | Sınıf ekler                       |
| <code>.removeClass("sinif")</code>                   | Sınıf kaldırır                    |
| <code>.toggleClass("sinif")</code>                   | Sınıf varsa kaldırır, yoksa ekler |
| <code>.show()</code> / <code>.hide()</code>          | Görünür / gizli yapar             |
| <code>.toggle()</code>                               | Görünürlük durumunu çevirir       |
| <code>.fadeIn(ms)</code> / <code>.fadeOut(ms)</code> | Animasyonla göster/gizle          |

## 2.4.3 Örnekler

### Örnek 2.4.1 — `.text()` ve `.html()` farkı

Amaç: İki metodun aynı içerik için farklı işleyişini göstermek.

```
<!-- ornekler/02-text-html.html -->
<div id="a"></div>
<div id="b"></div>

<script>
$("#a").text("<b>Kalın</b>");
$("#b").html("<b>Kalın</b>");
</script>
```

Çıktı: (yok — yan etki: `#a` öğesi sayfada `<b>Kalın</b>` metnini düz olarak gösterir; `#b` öğesi **Kalın** olarak kalın stilde gösterir)

### Örnek 2.4.2 — `.css()` ile dinamik renk değişimi

Amaç: Butona tıklandığında bir öğenin arka planını değiştirmek.

```
<!-- ornekler/02-css-renk.html -->
<div id="kutu" style="width:120px;height:60px;border:1px solid #ccc"></div>
<button id="btn">Renklendir</button>

<script>
$("#btn").click(function () {
    $("#kutu").css({
        background: "#EEDDFE",
        color: "#3C3489",
        textAlign: "center",
        lineHeight: "60px"
    });
    $("#kutu").text("Değişti");
});
</script>
```

Çıktı: (yok — yan etki: butona tıklandığında `#kutu` öğesinin arka planı ve metni değişir)

### Örnek 2.4.3 — Listeye dinamik öğe ekleme

Amaç: Input'tan okunan değeri listeye eklemek.

```
<!-- ornekler/02-liste-ekle.html -->
<input id="yeni" type="text" />
<button id="ekle">Ekle</button>
<ul id="liste"></ul>

<script>
$("#ekle").click(function () {
    const deger = $("#yeni").val().trim();
    if (deger === "") return;
    $("#liste").append("<li>" + deger + "</li>");
    $("#yeni").val("");
});
</script>
```

Çıktı: (yok — yan etki: her "Ekle" tıklamasıyla `#liste` öğesine yeni `<li>` eklenir, input temizlenir)

## 2.5 Bölüm Özeti

- jQuery, DOM manipülasyonunu sadeleştiren bir kütüphanedir; MVC projelerinde validation ve AJAX için zorunlu altyapıdır.
- Seçiciler CSS söz dizimini takip eder: `$("#id)`, `$(".sinif)`, `$("#etiket)`.
- Olay bağlama metotları: `.click()`, `.change()`, `.keyup()`, `.submit()`, `.hover()`.
- İçerik metotları: `.text()` (düz metin), `.html()` (HTML), `.val()` (form değeri).
- Stil metotları: `.css()`, `.addClass()`, `.toggle()`, `.fadeIn()`.
- Tüm jQuery kodu `$(document).ready(function () { ... })` içinde tanımlanmalıdır.

# Bölüm 3 — Nesne Yönelimli Programlama Temelleri

## 3.1 Genel Bakış

Nesne yönelimli programlama (OOP — Object-Oriented Programming), gerçek dünyadaki varlıkları yazılım nesneleri olarak modelleyen bir paradigmadır. Veri (durum) ve davranış (metotlar) tek bir birim olan **sınıf** içinde bir araya getirilir; bu sınıftan üretilen **nesne** kendi durumunu korur.

OOP'nin temel motivasyonu prosedürel programlamadaki üç sorunu çözmektir: **kod tekrarı**, **veri ile davranışın dağınık olması** ve **büyük kod tabanlarında bakım zorluğu**. Bu çözüm dört sütun üzerine kurulur: encapsulation (kapsülleme), inheritance (kalıtım), polymorphism (çok biçimlilik) ve abstraction (soyutlama).

## 3.2 Sınıf ve Nesne

### 3.2.1 Tanımlar

- Sınıf** (class): Nesnelerin nasıl üretileceğini belirleyen şablondur. Hangi alanlara (property) sahip olacaklarını, hangi davranışları (method) gerçekleştirebileceklerini tanımlar.
- Nesne** (object): Bir sınıftan `new` anahtar kelimesiyle üretilen, kendi durumuna sahip somut bir örnektir.
- Constructor**: Nesne oluşturulurken otomatik çalışan, başlangıç değerlerini ayarlayan özel metod.

### 3.2.2 Söz Dizimi

```
// Models/Araba.cs
public class Araba
{
    // Property — durum
    public string Marka { get; set; }
    public int Yil { get; set; }

    // Constructor
    public Araba(string marka, int yil)
    {
        Marka = marka;
        Yil = yil;
    }

    // Method — davranış
    public void Bilgi()
    {
        Console.WriteLine($"[Marka] ({Yil})");
    }
}
```

### 3.2.3 Davranış

Aynı sınıftan üretilen her nesne **kendi durumunu bağımsız tutar**. Bir nesnedeki property değişikliği diğer nesneyi etkilemez. Constructor yalnızca nesne oluşturulurken bir kez çalışır.

### 3.2.4 Örnek

#### Örnek 3.2.1 — Sınıftan nesne üretmek

Amaç: Aynı sınıftan iki bağımsız nesne oluşturmak.

```
// Program.cs
Araba a1 = new Araba("Toyota", 2023);
Araba a2 = new Araba("BMW", 2024);

a1.Bilgi();
a2.Bilgi();
```

Çıktı:

```
Toyota (2023)
BMW (2024)
```

## 3.3 Property ve Method

### 3.3.1 Property (Özellik)

Property, bir nesnenin durum bilgisini tutan, dışarıdan kontrollü erişim sunan üyedir. Otomatik property söz dizimi en yaygın kullanımdır:

```
public string Ad { get; set; } // okuma + yazma
public int Id { get; } // sadece okuma (yalnızca constructor'da set edilir)
public string Tip { get; private set; } // dışarıdan okunur, içeriden yazılır
```

### 3.3.2 Method (Metot)

Method, sınıfın gerçekleştirebildiği bir eylemi tanımlar. İmzası şöyledir:

```
[erişim] [donus_tipi] AdMetot([parametreler])
{
    // gövde
    return deger; // dönüş tipi void değilse
}
```

### 3.3.3 Örnek

#### Örnek 3.3.1 — Property ve method bir arada

Amaç: Bir öğrenci sınıfında not ekleme ve ortalama hesaplama davranışlarını uygulamak.

```
// Models/Ogrenci.cs
public class Ogrenci
{
    public string Ad { get; set; }
    public int OgrenciNo { get; }
    private List<int> _notlar = new();

    public Ogrenci(string ad, int no)
    {
        Ad = ad;
        OgrenciNo = no;
    }

    public void NotEkle(int not)
    {
        if (not < 0 || not > 100)
            throw new ArgumentOutOfRangeException(nameof(not));
        _notlar.Add(not);
    }

    public double Ortalama()
        => _notlar.Count == 0 ? 0 : _notlar.Average();
}
```

```
// Program.cs
var o = new Ogrenci("Ayşe", 101);
o.NotEkle(85);
o.NotEkle(92);
Console.WriteLine($"{o.Ad}: {o.Ortalama():F2}");
```

Çıktı:

## 3.4 Encapsulation (Kapsülleme)

### 3.4.1 Tanım

Encapsulation, bir sınıfın iç verisini (private alanlar) dışarıya kapatıp **kontrollü erişim** sağlamaktır. Property'ler ve metotlar bu erişimi yönetir; doğrudan alanlara erişim verilmez.

### 3.4.2 Erişim Belirleyiciler

| Belirleyici        | Erişim alanı                           |
|--------------------|----------------------------------------|
| public             | Her yerden                             |
| private            | Sadece sınıf içinden                   |
| protected          | Sınıf ve ondan türeyen alt sınıflardan |
| internal           | Aynı assembly içinden                  |
| protected internal | Aynı assembly + alt sınıflar           |

### 3.4.3 Örnek

#### Örnek 3.4.1 — Kapsüllenmiş banka hesabı

Amaç: Bakiyeye doğrudan erişimi engelleyip iş kurallarını metotlarda toplamak.

```
// Models/BankaHesabi.cs
public class BankaHesabi
{
    private decimal _bakiye;           // dışarıdan görünmez
    public decimal Bakiye => _bakiye;  // salt okunur dışarıya

    public BankaHesabi(decimal baslangic)
    {
        if (baslangic < 0)
            throw new ArgumentException("Başlangıç bakiyesi negatif olamaz");
        _bakiye = baslangic;
    }

    public void ParaYatir(decimal tutar)
    {
        if (tutar <= 0)
            throw new ArgumentException("Tutar pozitif olmalı");
        _bakiye += tutar;
    }

    public bool ParaCek(decimal tutar)
    {
        if (tutar > _bakiye) return false;
        _bakiye -= tutar;
        return true;
    }
}
```

```
// Program.cs
var h = new BankaHesabi(1000m);
h.ParaYatir(500m);
Console.WriteLine(h.Bakiye);
bool sonuc = h.ParaCek(2000m);
Console.WriteLine(sonuc);
```

Çıktı:

```
1500
False
```

### 3.4.4 Yaygın Hatalar ve Çözümleri

**Hata:** `public` alan kullanımı

```
// Hatalı - veri korumasız
public class Hesap
{
    public decimal Bakiye; // her yerden değiştirilebilir
}

var h = new Hesap();
h.Bakiye = -999999; // hiçbir kontrol yok
```

Neden hata: Dışarıdan rasgele atama mümkün olduğu için iş kuralları (negatif değer engeli, log tutma vb.) atlanır.

Düzeltilme: Alanı `private` yapın, erişim için property veya metod kullanın.

## 3.5 Inheritance (Kalıtım)

### 3.5.1 Tanım

Inheritance, bir sınıfın başka bir sınıfın üyelerini **miras almasıdır**. Ortak davranışlar üst sınıfta toplanır; alt sınıflar bunları kullanır, genişletir veya değiştirir.

### 3.5.2 Söz Dizimi

```
// Models/Calisan.cs
public class Calisan
{
    public string Ad { get; }
    protected decimal TemelMaas { get; set; }

    public Calisan(string ad, decimal maas)
    {
        Ad = ad;
        TemelMaas = maas;
    }

    public virtual decimal MaasHesapla() => TemelMaas;
}

// Models/Yazilimci.cs
public class Yazilimci : Calisan
{
    public string Dil { get; }

    public Yazilimci(string ad, decimal maas, string dil)
        : base(ad, maas)
    {
        Dil = dil;
    }

    public override decimal MaasHesapla()
        => TemelMaas * 1.3m;
}
```

Öğeler:

- `:` operatörü kalıtımı işaretler

- `base(...)` üst sınıf constructor'ını çağırır
- `virtual` anahtar kelimesi, metodun override edilebilir olduğunu belirtir
- `override` anahtar kelimesi, alt sınıfta metodu yeniden tanımlar
- `protected` alanlar alt sınıflar tarafından erişilebilir

### 3.5.3 Örnek

#### Örnek 3.5.1 — Üst ve alt sınıf birlikte kullanımı

Amaç: Genel `Calisan` sınıfını miras alan `Yazilimci`'nin farklı maaş hesaplamasını göstermek.

```
// Program.cs
Calisan c1 = new Calisan("Ahmet", 20000m);
Calisan c2 = new Yazilimci("Ayşe", 20000m, "C#");

Console.WriteLine(c1.MaasHesapla());
Console.WriteLine(c2.MaasHesapla());
```

Çıktı:

```
20000
26000,0
```

## 3.6 Polymorphism (Çok Biçimlilik)

### 3.6.1 Tanım

Polymorphism, **üst sınıf referansının alt sınıf nesnelerini tutması ve doğru metodun çalışma zamanında seçilmesi** anlamına gelir. `virtual / override` mekanizması veya `interface`'ler aracılığıyla uygulanır.

### 3.6.2 Örnek

#### Örnek 3.6.1 — Liste içinde farklı tipler

Amaç: Aynı üst tipte tutulan farklı alt sınıf nesnelerinin polimorfik davranışını göstermek.

```
// Program.cs
List<Calisan> personel = new()
{
    new Calisan("Ahmet", 20000m),
    new Yazilimci("Ayşe", 20000m, "C#"),
    new Yazilimci("Mehmet", 22000m, "Python")
};

decimal toplam = 0;
foreach (Calisan c in personel)
{
    decimal m = c.MaasHesapla(); // runtime'da doğru override seçilir
    toplam += m;
    Console.WriteLine($"{c.Ad}: {m}");
}
Console.WriteLine($"Toplam: {toplam}");
```

Çıktı:

```
Ahmet: 20000
Ayşe: 26000,0
Mehmet: 28600,0
Toplam: 74600,0
```

### 3.6.3 Tip Kontrolü — `is` ve `as`

```
// Program.cs
foreach (var c in personel)
{
    if (c is Yazilimci dev)
        Console.WriteLine($"{dev.Ad} → {dev.Dil}");
}
```

Çıktı:

```
Ayşe → C#
Mehmet → Python
```

## 3.7 Abstraction (Soyutlama)

### 3.7.1 Tanım

Abstraction, "ne yaptığını" gösterip "nasıl yaptığını" gizlemektir. İki ana mekanizma vardır:

- **abstract class** : Doğrudan örneklenemez, alt sınıflara şablon sunar. Bazı metotları soyut (gövdesiz) olabilir.
- **interface** : Yalnızca üye imzalarını tanımlar. Bir sınıf birden fazla interface uygulayabilir.

### 3.7.2 Söz Dizimi

```
// Models/Sekil.cs
public abstract class Sekil
{
    public string Renk { get; }
    protected Sekil(string renk) => Renk = renk;

    public abstract double Alan();          // gövdesiz — zorunlu uygulama
    public abstract double Cevre();
}

// Models/Daire.cs
public class Daire : Sekil
{
    private double _r;
    public Daire(double r, string renk) : base(renk) => _r = r;

    public override double Alan()  => Math.PI * _r * _r;
    public override double Cevre() => 2 * Math.PI * _r;
}
```

### 3.7.3 Interface

```
// Models/IKaydedilebilir.cs
public interface IKaydedilebilir
{
    void Kaydet();
    bool Sil(int id);
}

// Models/Kullanici.cs
public class Kullanici : IKaydedilebilir
{
    public string Eposta { get; set; }

    public void Kaydet() { /* veritabanına yaz */ }
    public bool Sil(int id) { /* ... */ return true; }
}
```

### 3.7.4 Örnek

### Örnek 3.7.1 — Soyut sınıftan polimorfik kullanım

Amaç: Farklı şekillerin ortak `Sekil` tipinde tutulup alanlarının hesaplanması.

```
// Program.cs
Sekil[] sekiller =
{
    new Daire(5, "kırmızı"),
    new Daire(3, "mavi")
};

foreach (var s in sekiller)
    Console.WriteLine($"{s.Renk}: {s.Alan():F2}");
```

Çıktı:

```
kırmızı: 78,54
mavi: 28,27
```

## 3.8 OOP vs Prosedürel Programlama

| Kriter      | Prosedürel                      | Nesne Yönelimli                |
|-------------|---------------------------------|--------------------------------|
| Yapı        | Fonksiyonlar sırayla çalışır    | Nesneler birbiriyle etkileşir  |
| Veri        | Global / yerel değişkenler      | Nesne içinde kapsüllenmiş      |
| Kod tekrarı | Yüksek                          | Kalıtım sayesinde düşük        |
| Bakım       | Büyükünce zorlaşır              | Modüler — kolay                |
| Test        | Bağımlılıklar zorlaştırır       | İzole sınıflar test edilebilir |
| Genişletme  | Mevcut kodu değiştirmek gerekir | Polymorphism ile güvenli       |

## 3.9 Mini Sözlük

- **Sınıf** (class): Nesne şablonu.
- **Nesne** (object): Sınıfın somut örneği.
- **Property** (özellik): Nesnenin durum bilgisi.
- **Method** (metot): Nesnenin davranışı.
- **Encapsulation** (kapsülleme): İç veriyi gizleyip kontrollü erişim sağlama.
- **Inheritance** (kalıtım): Bir sınıfın başka bir sınıfın üyelerini miras alması.
- **Polymorphism** (çok biçimlilik): Üst sınıf referansı üzerinden alt sınıf davranışının çalışması.
- **Abstraction** (soyutlama): Karmaşıklığı gizleyip sade arayüz sunma.

## 3.10 Bölüm Özeti

- OOP'nin dört sütunu: encapsulation, inheritance, polymorphism, abstraction.
- Sınıf bir şablondur, nesne onun somut örneğidir.
- Property'ler durum bilgisini, metotlar davranışı temsil eder.
- `private` alanlar dışarıdan gizlenir; iş kuralları metotlarda korunur.
- Kalıtımda `:` operatörü ve `base(...)` çağrısı temel söz dizimidir.
- `virtual / override` runtime polymorphism'in temelidir.
- `abstract class` ve `interface` soyutlamanın iki ana aracıdır.

# Bölüm 4 — SOLID İlkeleri

## 4.1 Genel Bakış

SOLID, Robert C. Martin (Uncle Bob) tarafından sistematize edilen beş yazılım tasarım ilkesidir. Amaçları **bakımı kolay, genişletmeye açık, test edilebilir** kod yazmaktır.

- **S** — Single Responsibility Principle
- **O** — Open/Closed Principle
- **L** — Liskov Substitution Principle
- **I** — Interface Segregation Principle
- **D** — Dependency Inversion Principle

## 4.2 Single Responsibility Principle (SRP)

### 4.2.1 Tanım

Bir sınıfın **yalnızca tek bir değişme sebebi olmalıdır**. Her sınıf tek bir işi yapmalıdır.

### 4.2.2 Anti-pattern Örneği

#### Örnek 4.2.1 — Aşırı yüklenmiş sınıf

```
// Models/Rapor.cs
public class Rapor
{
    public string Baslik { get; set; }

    public string IcerikOlustur() { /* veri toplama */ return ""; }
    public void DosyayaKaydet() { /* File.WriteAllText */ }
    public void Yazdir() { /* yazıcıya gönder */ }
    public void EpostalaGonder() { /* SMTP çağrısı */ }
}
```

Bu sınıfın **dört değişim sebebi** vardır: veri kuralı, dosya formatı, yazıcı API'si, e-posta sağlayıcı. Herhangi birinin değişmesi tüm sınıfı etkiler.

### 4.2.3 SRP Uyumlu Hali

#### Örnek 4.2.2 — Sorumluluk dağıtımı

```
// Models/Rapor.cs
public class Rapor
{
    public string Baslik { get; set; }
    public string IcerikOlustur() => "...";
}

// Services/RaporKaydedici.cs
public class RaporKaydedici
{
    public void Kaydet(Rapor r) => File.WriteAllText($"{r.Baslik}.txt", r.IcerikOlustur());
}

// Services/RaporYazici.cs
public class RaporYazici
{
    public void Yazdir(Rapor r) { /* yazıcı API'si */ }
}
```

Her sınıfın tek bir değişim sebebi vardır.

## 4.3 Open/Closed Principle (OCP)

---

### 4.3.1 Tanım

Sınıflar **geniştirmeye açık, değişikliğe kapalı** olmalıdır. Yeni özellik eklerken mevcut sınıfı değiştirmek yerine yeni sınıf eklenmelidir.

### 4.3.2 Anti-pattern Örneği

```
// Services/OdemeIslemci.cs
public class OdemeIslemci
{
    public void Ode(string tip, decimal tutar)
    {
        if (tip == "kredi")
            { /* kredi kartı işlemi */ }
        else if (tip == "nakit")
            { /* nakit işlemi */ }
        // Yeni ödeme türü → bu metodu değiştirmek gerekir
    }
}
```

Her yeni ödeme yöntemi mevcut metoda dokunmayı gerektirir; bu da mevcut testlerin geçersizleşmesi anlamına gelir.

### 4.3.3 OCP Uyumlu Hali

#### Örnek 4.3.1 — Polimorfizm ile genişletme

```
// Services/IOdeme.cs
public interface IOdeme
{
    void OdemeYap(decimal tutar);
}

// Services/KrediKarti.cs
public class KrediKarti : IOdeme
{
    public void OdemeYap(decimal tutar)
        => Console.WriteLine($"Kart: {tutar}");
}

// Services/Kripto.cs — YENİ
public class Kripto : IOdeme
{
    public void OdemeYap(decimal tutar)
        => Console.WriteLine($"BTC: {tutar}");
}
```

Yeni ödeme türü eklerken **mevcut kod hiç değişmez**.

## 4.4 Liskov Substitution Principle (LSP)

---

### 4.4.1 Tanım

Alt sınıf nesnesi, üst sınıf nesnesinin yerine kullanıldığında programın davranışı bozulmamalıdır.

### 4.4.2 Anti-pattern Örneği

```
// Models/Kus.cs
public class Kus
{
    public virtual void Uc() => Console.WriteLine("Uçuyorum");
}

// Models/Penguen.cs
public class Penguen : Kus
{
    public override void Uc()
        => throw new NotSupportedException("Penguenler uçamaz!");
}
```

Penguen, Kus'un yerine konulduğunda program patlar — LSP ihlali.

### 4.4.3 LSP Uyumlu Hiyerarşi

#### Örnek 4.4.1 — Davranışı interface ile ayırma

```
// Models/Kus.cs
public abstract class Kus
{
    public abstract void Hareket();
}

// Models/IUcabilir.cs
public interface IUcabilir
{
    void Uc();
}

// Models/Serce.cs
public class Serce : Kus, IUcabilir
{
    public override void Hareket() => Uc();
    public void Uc() => Console.WriteLine("Uçuyorum");
}

// Models/Penguen.cs
public class Penguen : Kus // IUcabilir uygulamıyor
{
    public override void Hareket() => Console.WriteLine("Yüzüyorum");
}
```

## 4.5 Interface Segregation Principle (ISP)

### 4.5.1 Tanım

Bir sınıf, kullanmayacağı metotları uygulamak zorunda bırakılmamalıdır. Geniş interface'leri **küçük, odaklı** interface'lere bölün.

### 4.5.2 Anti-pattern Örneği

```
// Şişirilmiş interface
public interface ICalisan
{
    void Calis();
    void Ye();
    void ToplantiyaKatil();
}

public class Robot : ICalisan
{
    public void Calis() { /* ok */ }
    public void Ye() => throw new NotImplementedException();
    public void ToplantiyaKatil() => throw new NotImplementedException();
}
```

Robot, anlamsız metotları uygulamak zorunda kalır.

### 4.5.3 ISP Uyumlu Hali

#### Örnek 4.5.1 — Küçük interface'lere bölme

```
// Services/ICalisan.cs
public interface ICalisan { void Calis(); }

// Services/IYiyebilen.cs
public interface IYiyebilen { void Ye(); }

// Services/Robot.cs
public class Robot : ICalisan
{
    public void Calis() { /* ok */ }
}

// Services/Insan.cs
public class Insan : ICalisan, IYiyebilen
{
    public void Calis() { /* ok */ }
    public void Ye() { /* ok */ }
}
```

## 4.6 Dependency Inversion Principle (DIP)

### 4.6.1 Tanım

Üst seviye modüller alt seviye modüllere bağımlı olmamalıdır. Her ikisi de **soyutlamalara** (interface) bağımlı olmalıdır.

### 4.6.2 Anti-pattern Örneği

```
// Services/SiparisServisi.cs
public class SiparisServisi
{
    private MySqlVeritabani _db = new MySqlVeritabani(); // somut bağımlılık

    public void Kaydet(Siparis s) => _db.Kaydet(s);
}
```

PostgreSQL'e geçiş, tüm sınıfı değiştirmeyi gerektirir.

### 4.6.3 DIP Uyumlu Hali

#### Örnek 4.6.1 — Constructor injection ile interface bağımlılığı

```
// Services/IVeritabani.cs
public interface IVeritabani
{
    void Kaydet(Siparis s);
}

// Services/MySqlDb.cs
public class MySqlDb : IVeritabani
{
    public void Kaydet(Siparis s) { /* ... */ }
}

// Services/SiparisServisi.cs
public class SiparisServisi
{
    private readonly IVeritabani _db;

    public SiparisServisi(IVeritabani db) // bağımlılık dışarıdan enjekte edilir
    {
        _db = db;
    }

    public void Kaydet(Siparis s) => _db.Kaydet(s);
}
```

Bu yapı ASP.NET Core'un DI Container'ı ile birlikte (bkz. §14) doğal biçimde uygulanır.

## 4.7 Anti-Pattern Tablosu

| Anti-pattern                                    | İdiyomatik Karşılığı                    | Neden Daha İyi                        |
|-------------------------------------------------|-----------------------------------------|---------------------------------------|
| Tek sınıfta çok sorumluluk                      | Her sorumluluk için ayrı sınıf (SRP)    | Bakım kolaylığı, test edilebilirlik   |
| <code>if/else</code> ile tip seçimi             | Polimorfizm + interface (OCP)           | Yeni tip eklerken mevcut kod değişmez |
| Üst sınıfın davranışını bozan alt sınıf         | Davranışı interface ile ayırma (LSP)    | Beklenmedik exception'lar önlenir     |
| Tek büyük interface                             | Küçük, odaklı interface'ler (ISP)       | Sınıflar gereksiz metot uygulamaz     |
| <code>new ConcreteSınıf()</code> ile bağımlılık | Constructor injection + interface (DIP) | Test ve değiştirilebilirlik           |

## 4.8 Bölüm Özeti

- SRP — bir sınıfın tek bir değişim sebebi olmalı.
- OCP — yeni özellik için mevcut kodu değiştirme, yeni sınıf ekle.
- LSP — alt sınıf, üst sınıfın yerine sorunsuz geçebilmeli.
- ISP — interface'ler küçük ve odaklı olmalı.
- DIP — somut sınıflara değil, interface'lere bağımlı ol.
- Tüm ilkeler birlikte uygulandığında modüler, sürdürülebilir, test edilebilir tasarım ortaya çıkar.

# Bölüm 5 — C# Söz Dizimi ve Temel Yapılar

## 5.1 Değişkenler ve Veri Tipleri

### 5.1.1 Genel Bakış

C#, **statik tipli** bir dildir: her değişkenin tipi derleme zamanında bellidir. Tipler iki kategoriye ayrılır:

- **Value types** (değer tipleri): `int`, `double`, `bool`, `char`, `struct`, `enum`. Değerleri doğrudan tutar.
- **Reference types** (referans tipleri): `string`, `class`, `interface`, `array`. Bellekteki nesneye referans tutar.

### 5.1.2 Temel Veri Tipleri

| Tip                  | Bayt | Aralık / Açıklama                      | Örnek değer              |
|----------------------|------|----------------------------------------|--------------------------|
| <code>byte</code>    | 1    | 0 ile 255 arası                        | <code>42</code>          |
| <code>short</code>   | 2    | $\pm 32.767$                           | <code>1000</code>        |
| <code>int</code>     | 4    | $\pm 2,1$ milyar                       | <code>42</code>          |
| <code>long</code>    | 8    | $\pm 9,2 \times 10^{18}$               | <code>85_000_000L</code> |
| <code>float</code>   | 4    | ~7 basamak hassasiyet                  | <code>3.14f</code>       |
| <code>double</code>  | 8    | ~15 basamak hassasiyet                 | <code>3.14</code>        |
| <code>decimal</code> | 16   | ~28 basamak hassasiyet (parasal)       | <code>9.99m</code>       |
| <code>bool</code>    | 1    | <code>true</code> / <code>false</code> | <code>true</code>        |
| <code>char</code>    | 2    | Tek Unicode karakter                   | <code>'A'</code>         |
| <code>string</code>  | -    | Karakter dizisi (referans tipi)        | <code>"Merhaba"</code>   |

### 5.1.3 Söz Dizimi

```
// Program.cs
int yas = 25;
long nufus = 85_000_000L;
double agirlik = 72.5;
decimal fiyat = 149.99m; // m suffixi decimal için zorunlu
string ad = "Ayşe";
char harf = 'A';
bool aktif = true;

// var - derleyici tipi çıkarır
var soyad = "Yılmaz"; // string olarak inferred edilir
var sayi = 42; // int olarak inferred edilir
```

### 5.1.4 Tip Dönüşümü

C# iki tür dönüşüm sunar: **örtük** (implicit) ve **açık** (explicit). Veri kaybı riski olmayan dönüşümler örtüktür; diğerleri cast veya `Parse` / `Convert` ile yapılır.

#### Örnek 5.1.1 — String'den sayıya dönüşüm

Amaç: Kullanıcı girdisini sayıya çevirmek, güvenli (`TryParse`) ve güvensiz (`Parse`) yöntemleri göstermek.

```
// Program.cs
string girdi1 = "42";
int sayi1 = int.Parse(girdi1);
Console.WriteLine(sayi1);

string girdi2 = "abc";
bool ok = int.TryParse(girdi2, out int sayi2);
Console.WriteLine($"Başarılı mı: {ok}, Değer: {sayi2}");
```

Çıktı:

```
42
Başarılı mı: False, Değer: 0
```

### 5.1.5 String Interpolation

```
// Program.cs
string ad = "Ayşe";
int yas = 22;

string mesaj = $"Merhaba {ad}, yaşıınız: {yas}";
Console.WriteLine(mesaj);
```

Çıktı:

```
Merhaba Ayşe, yaşıınız: 22
```

## 5.2 Karar Yapıları

### 5.2.1 if / else if / else

```
// Program.cs
int not = 78;
string harf;

if (not >= 90) harf = "AA";
else if (not >= 80) harf = "BB";
else if (not >= 70) harf = "CC";
else if (not >= 60) harf = "DD";
else harf = "FF";

Console.WriteLine(harf);
```

Çıktı:

```
CC
```

### 5.2.2 Switch Expression (C# 8+)

```
// Program.cs
int not = 78;

string harf = not switch
{
    >= 90 => "AA",
    >= 80 => "BB",
    >= 70 => "CC",
    >= 60 => "DD",
    _     => "FF"
};

Console.WriteLine(harf);
```

Çıktı:

```
CC
```

### 5.2.3 Pattern Matching ( is )

```
// Program.cs
object o = 42;

if (o is int n && n > 0)
    Console.WriteLine($"Pozitif int: {n}");
```

Çıktı:

```
Pozitif int: 42
```

## 5.3 Döngüler

### 5.3.1 for Döngüsü

```
// Program.cs
int toplam = 0;
for (int i = 1; i <= 5; i++)
    toplam += i;

Console.WriteLine(toplam);
```

Çıktı:

```
15
```

### 5.3.2 foreach Döngüsü

```
// Program.cs
string[] isimler = { "Ali", "Veli", "Ayşe" };

foreach (string isim in isimler)
    Console.WriteLine(isim.ToUpper());
```

Çıktı:

```
ALI
VELI
AYŞE
```

### 5.3.3 while ve do-while

```
// Program.cs
int x = 1;
while (x <= 5)
{
    Console.Write(x + " ");
    x++;
}
Console.WriteLine();
```

Çıktı:

```
1 2 3 4 5
```

## 5.4 Diziler ve Koleksiyonlar

### 5.4.1 Dizi (Array)

Sabit boyutlu, aynı tipte elemanları tutar.

```
// Program.cs
int[] notlar = { 85, 92, 78, 95, 60 };

Console.WriteLine(notlar[0]);    // ilk
Console.WriteLine(notlar[1]);    // son (C# 8+ index)

Array.Sort(notlar);
Console.WriteLine(string.Join(", ", notlar));
```

Çıktı:

```
85
60
60, 78, 85, 92, 95
```

### 5.4.2 List<T> — Dinamik Liste

Boyut otomatik büyür, eleman ekleme/çıkarma kolaydır.

```
// Program.cs
var ogrenciler = new List<string> { "Ali", "Veli" };

ogrenciler.Add("Ayşe");
ogrenciler.Insert(0, "Zeynep");
ogrenciler.Remove("Veli");

Console.WriteLine(string.Join(", ", ogrenciler));
Console.WriteLine($"Sayı: {ogrenciler.Count}");
Console.WriteLine($"Ali var mı: {ogrenciler.Contains("Ali")}");
```

Çıktı:

```
Zeynep, Ali, Ayşe
Sayı: 3
Ali var mı: True
```

### 5.4.3 Dictionary<TKey, TValue> — Anahtar-Değer

```
// Program.cs
var puanlar = new Dictionary<string, int>
{
    ["Ali"] = 85,
    ["Ayşe"] = 95
};
puanlar["Veli"] = 70;

if (puanlar.TryGetValue("Ali", out int p))
    Console.WriteLine($"Ali: {p}");
```

Çıktı:

```
Ali: 85
```

## 5.5 Sık Kullanılan String Metotları

| Metot                                       | Açıklama                                                               | Örnek                                                           |
|---------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>.ToUpper()</code>                     | Büyük harfe çevirir                                                    | <code>"abc".ToUpper()</code> → <code>"ABC"</code>               |
| <code>.ToLower()</code>                     | Küçük harfe çevirir                                                    | <code>"ABC".ToLower()</code> → <code>"abc"</code>               |
| <code>.Trim()</code>                        | Baştaki ve sondaki boşlukları siler                                    | <code>" abc ".Trim()</code> → <code>"abc"</code>                |
| <code>.Contains(s)</code>                   | İçinde <code>s</code> var mı?                                          | <code>"abcdef".Contains("cd")</code> → <code>true</code>        |
| <code>.StartsWith(s)</code>                 | <code>s</code> ile başlıyor mu?                                        | <code>"abc".StartsWith("a")</code> → <code>true</code>          |
| <code>.Replace(eski, yeni)</code>           | <code>eski</code> karakteri/dizgisini <code>yeni</code> ile değiştirir | <code>"abc".Replace("b", "X")</code> → <code>"aXc"</code>       |
| <code>.Split(ayrac)</code>                  | Diziye böler                                                           | <code>"a,b,c".Split(',')</code> → <code>["a","b","c"]</code>    |
| <code>.Substring(baslangic, uzunluk)</code> | Alt dize alır                                                          | <code>"abcdef".Substring(1,3)</code> → <code>"bcd"</code>       |
| <code>.Length</code>                        | Karakter sayısı (property)                                             | <code>"abc".Length</code> → <code>3</code>                      |
| <code>string.IsNullOrEmpty(s)</code>        | <code>null</code> veya boş mu?                                         | -                                                               |
| <code>string.Join(ayrac, koleksiyon)</code> | Liste elemanlarını birleştirir                                         | <code>string.Join(", ", ["a","b"])</code> → <code>"a, b"</code> |

## 5.6 Metotlar

### 5.6.1 Söz Dizimi

```
// Services/Hesap.cs
public class Hesap
{
    // void – dönüş yok
    public void Selamla(string isim)
        => Console.WriteLine($"Merhaba {isim}");

    // Dönüş değerli
    public int Topla(int a, int b) => a + b;

    // Varsayılan parametre
    public string Karsila(string isim, string dil = "tr")
        => dil == "tr" ? $"Merhaba {isim}" : $"Hello {isim}";

    // params – değişken sayıda parametre
    public int ToplaCok(params int[] sayilar) => sayilar.Sum();

    // out – birden çok değer döndürme
    public bool Bol(int a, int b, out int sonuc)
    {
        if (b == 0) { sonuc = 0; return false; }
        sonuc = a / b;
        return true;
    }
}
```

## 5.6.2 Örnek

### Örnek 5.6.1 — `out` ile güvenli bölme

```
// Program.cs
var h = new Hesap();

if (h.Bol(10, 3, out int sonuc))
    Console.WriteLine($"Sonuç: {sonuc}");

if (!h.Bol(10, 0, out int hata))
    Console.WriteLine("Sıfıra bölme!");

Console.WriteLine(h.ToplaCok(1, 2, 3, 4, 5));
```

Çıktı:

```
Sonuç: 3
Sıfıra bölme!
15
```

## 5.7 LINQ — Language Integrated Query

LINQ, koleksiyonlar üzerinde sorgulama yapmak için C#'a entegre edilmiş bir API'dir. SQL benzeri zincirleme metotlar sunar.

### 5.7.1 Yaygın LINQ Metotları

| Metot                                                                                 | Açıklama                                               |
|---------------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>Where(fn)</code>                                                                | Koşulu sağlayanları filtreler                          |
| <code>Select(fn)</code>                                                               | Her elemanı dönüştürür                                 |
| <code>OrderBy(fn)</code>                                                              | Artan sırada sıralar                                   |
| <code>OrderByDescending(fn)</code>                                                    | Azalan sırada sıralar                                  |
| <code>First()</code> / <code>FirstOrDefault()</code>                                  | İlk eleman                                             |
| <code>Count()</code>                                                                  | Eleman sayısı                                          |
| <code>Sum()</code> , <code>Average()</code> , <code>Max()</code> , <code>Min()</code> | Agregat işlemler                                       |
| <code>Any(fn)</code>                                                                  | En az bir eleman koşulu sağlıyor mu?                   |
| <code>All(fn)</code>                                                                  | Tüm elemanlar koşulu sağlıyor mu?                      |
| <code>ToList()</code>                                                                 | Sonucu <code>List&lt;T&gt;</code> olarak somutlaştırır |

### 5.7.2 Örnek

#### Örnek 5.7.1 — Zincirleme LINQ sorgusu

Amaç: Geçen öğrencileri büyükten küçüğe sıralayıp etiketlemek.

```
// Program.cs
var notlar = new List<int> { 85, 42, 91, 60, 73 };

var gecenler = notlar
    .Where(n => n >= 60)
    .OrderByDescending(n => n)
    .Select(n => $" {n} ✓")
    .ToList();

Console.WriteLine(string.Join(", ", gecenler));
Console.WriteLine($"Ortalama: {notlar.Average():F2}");
Console.WriteLine($"Geçen sayısı: {gecenler.Count}");
```

Çıktı:

```
91 ✓, 85 ✓, 73 ✓, 60 ✓
Ortalama: 70,20
Geçen sayısı: 4
```

## 5.8 Bölüm Özeti

- C# statik tiplidir; tipler value (kopya) ve reference (referans) olmak üzere iki kategoride.
- Parasal değerler için `decimal` (suffix `m`) kullanılır; `double` yuvarlama hatası verir.
- `var` derleyici tip çıkarımıdır; tip belli olduğu için statik tiplilik bozulmaz.
- Karar: `if/else`, `switch`, switch expression, pattern matching.
- Döngü: `for`, `foreach`, `while`, `do-while`.
- Koleksiyonlar: `List<T>` (dinamik), `Dictionary<K,V>` (anahtar-değer), `array` (sabit boyut).
- `string` metotları arasında en kritikleri `ToLower`, `Contains`, `Trim`, `Split`, `Substring`.
- LINQ, koleksiyon sorgulamasını okunabilir zincirleme metotlarla yapar.

# Bölüm 6 — C# Sınıfları ve OOP Uygulaması

## 6.1 Sınıf Anatomisi

### 6.1.1 Yapı Taşları

Bir C# sınıfı şu bölümlerden oluşur:

1. **Property** — durum bilgisi
2. **Field** — sınıf içi değişken (genelde `private`)
3. **Constructor** — başlangıç değerlerini ayarlayan özel metot
4. **Method** — sınıfın davranışları
5. **Erişim belirleyiciler** — `public`, `private`, `protected`, `internal`

### 6.1.2 Tam Sınıf Örneği

#### Örnek 6.1.1 — Öğrenci yönetim sınıfı

Amaç: Tüm sınıf yapı taşlarını birleştiren bir model.

```
// Models/Ogrenci.cs
public class Ogrenci
{
    // Property'ler
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int OgrenciNo { get; } // salt-okunur

    // Field
    private List<int> _notlar = new(); // private alan

    // Constructor
    public Ogrenci(string ad, string soyad, int no)
    {
        Ad = ad;
        Soyad = soyad;
        OgrenciNo = no;
    }

    // Davranış metotları
    public void NotEkle(int not)
    {
        if (not < 0 || not > 100)
            throw new ArgumentOutOfRangeException(nameof(not));
        _notlar.Add(not);
    }

    public double Ortalama()
        => _notlar.Count == 0 ? 0 : _notlar.Average();

    public string HarfNotu() => Ortalama() switch
    {
        >= 90 => "AA",
        >= 80 => "BB",
        >= 70 => "CC",
        >= 60 => "DD",
        _ => "FF"
    };

    public override string ToString()
        => $"[{OgrenciNo}] {Ad} {Soyad} — Ort:{Ortalama():F1} ({HarfNotu()})";
}
```

```
// Program.cs
var o = new Ogrenci("Ayşe", "Yılmaz", 101);
o.NotEkle(85);
o.NotEkle(92);
o.NotEkle(78);

Console.WriteLine(o);
```

Çıktı:

```
[101] Ayşe Yılmaz — Ort:85,0 (BB)
```

## 6.2 Kalıtım ve Polymorphism — Çalışan Senaryosu

### 6.2.1 Üst Sınıf

```
// Models/Calisan.cs
public class Calisan
{
    public string Ad { get; }
    public string Sicil { get; }
    protected decimal TemelMaas { get; set; }

    public Calisan(string ad, string sicil, decimal maas)
    {
        Ad = ad;
        Sicil = sicil;
        TemelMaas = maas;
    }

    public virtual decimal MaasHesapla() => TemelMaas;

    public virtual string Bilgi()
        => $"[{Sicil}] {Ad} — Maaş: {MaasHesapla():C2}";
}
```

### 6.2.2 Alt Sınıflar

```
// Models/YazilimGelistirici.cs
public class YazilimGelistirici : Calisan
{
    public string ProgramlamaDili { get; }

    public YazilimGelistirici(string ad, string sicil, decimal maas, string dil)
        : base(ad, sicil, maas)
    {
        ProgramlamaDili = dil;
    }

    public override decimal MaasHesapla()
        => TemelMaas * 1.3m; // %30 teknoloji zammı

    public override string Bilgi()
        => base.Bilgi() + $" [{ProgramlamaDili}]";
}

// Models/Mudur.cs
public class Mudur : Calisan
{
    public int TakimSayisi { get; }

    public Mudur(string ad, string sicil, decimal maas, int takim)
        : base(ad, sicil, maas)
    {
        TakimSayisi = takim;
    }

    public override decimal MaasHesapla()
        => TemelMaas + (TakimSayisi * 2_000m); // ekip primi
}
```

### 6.2.3 Polimorfik Kullanım

#### Örnek 6.2.1 — Üst sınıf listesi ile alt sınıf nesnelerini yönetme

```
// Program.cs
var personel = new List<Calisan>
{
    new YazilimGelistirici("Ayşe", "Y001", 20000m, "C#"),
    new YazilimGelistirici("Kemal", "Y002", 22000m, "Python"),
    new Mudur("Mehmet", "M001", 30000m, 3)
};

foreach (Calisan c in personel)
    Console.WriteLine(c.Bilgi());

decimal toplamBordro = personel.Sum(c => c.MaasHesapla());
Console.WriteLine($"Toplam bordro: {toplamBordro:C2}");

// Tip kontrolü
foreach (var c in personel)
{
    if (c is YazilimGelistirici dev)
        Console.WriteLine($"{dev.Ad} → {dev.ProgramlamaDili}");
}
```

Çıktı (kültür ayarı `tr-TR` varsayılır):

```
[Y001] Ayşe — Maaş: ₺26.000,00 [C#]  
[Y002] Kemal — Maaş: ₺28.600,00 [Python]  
[M001] Mehmet — Maaş: ₺36.000,00
```

Toplam bordro: ₺90.600,00

Ayşe → C#

Kemal → Python

## 6.3 Abstract Class ve Interface — Kütüphane Senaryosu

### 6.3.1 Abstract Temel Sınıf

```
// Models/Materyal.cs  
public abstract class Materyal  
{  
    public Guid Id { get; } = Guid.NewGuid();  
    public string Baslik { get; }  
    public int Yil { get; }  
    private bool _oduncta;  
    public bool Oduncta => _oduncta;  
  
    protected Materyal(string baslik, int yil)  
    {  
        Baslik = baslik;  
        Yil = yil;  
    }  
  
    public abstract string Tur { get; }  
    public abstract string KisaBilgi();  
  
    public bool OduncVer()  
    {  
        if (_oduncta) return false;  
        _oduncta = true;  
        return true;  
    }  
  
    public void IadeAl() => _oduncta = false;  
  
    public override string ToString()  
        => $"[{Tur}] {Baslik} ({Yil}) — {(Oduncta ? "Ödünçte" : "rafta")}";  
}
```

### 6.3.2 Türeyen Somut Sınıflar

```
// Models/Kitap.cs
public class Kitap : Materyal
{
    public string Yazar { get; }
    public int SayfaSayisi { get; }

    public Kitap(string baslik, string yazar, int yil, int sayfa)
        : base(baslik, yil)
    {
        Yazar = yazar;
        SayfaSayisi = sayfa;
    }

    public override string Tur => "Kitap";

    public override string KisaBilgi()
        => $"{Baslik} / {Yazar} ({SayfaSayisi} sf.)";
}

// Models/Dergi.cs
public class Dergi : Materyal
{
    public string YayinEvi { get; }
    public int Sayi { get; }

    public Dergi(string baslik, string yayinevi, int yil, int sayi)
        : base(baslik, yil)
    {
        YayinEvi = yayinevi;
        Sayi = sayi;
    }

    public override string Tur => "Dergi";

    public override string KisaBilgi()
        => $"{Baslik} - Sayı:{Sayi} / {YayinEvi}";
}
```

### 6.3.3 Polimorfik Kullanım

#### Örnek 6.3.1 — Tek koleksiyonda farklı materyal türleri

```
// Program.cs
var koleksiyon = new List<Materyal>
{
    new Kitap("Temiz Kod", "Robert Martin", 2008, 431),
    new Kitap("C# Derinlemesine", "Jon Skeet", 2019, 528),
    new Dergi("Bilim Teknik", "TÜBİTAK", 2024, 678)
};

foreach (var m in koleksiyon)
    Console.WriteLine(m); // ToString() override çağrılır

// Sadece kitapları getir
var kitaplar = koleksiyon.OfType<Kitap>().ToList();
Console.WriteLine($"{nKitap sayısı: {kitaplar.Count}");

// Ödünç ver
koleksiyon[0].OduncVer();
Console.WriteLine($"{n{koleksiyon[0]}}");
```

Çıktı:

```
[Kitap] Temiz Kod (2008) – rafta
[Kitap] C# Derinlemesine (2019) – rafta
[Dergi] Bilim Teknik (2024) – rafta
```

Kitap sayısı: 2

```
[Kitap] Temiz Kod (2008) – ödünçte
```

## 6.4 Bölüm Özeti

---

- Bir sınıf property, field, constructor ve metotları bir arada barındırır.
  - `private readonly` field, dışarıdan değiştirilemeyen güvenli içsel durum sağlar.
  - `virtual` ve `override` runtime polimorfizmin anahtar kelimeleridir.
  - `abstract class` ortak şablon + zorunlu metot imzaları sunar; doğrudan örneklenemez.
  - `interface` saf sözleşmedir; bir sınıf birden fazla interface uygulayabilir.
  - Üst sınıf referansı ( `List<Calisan>` ) üzerinde alt sınıf nesneleri tutulup polimorfik çağrılar yapılır.
-

# Bölüm 7 — İlişkisel Veritabanı Tablo İlişkileri

## 7.1 Genel Bakış

İlişkisel veritabanlarında veriler tablolarda tutulur; **foreign key** (yabancı anahtar) sütunları aracılığıyla tablolar birbirine bağlanır. Üç temel ilişki türü vardır:

- **One-to-One (1—1)**: Her kayıt karşı tabloda yalnızca bir kayıtle eşleşir
- **One-to-Many (1—N)**: Bir kayıt karşı tabloda birden fazla kayıtle eşleşir
- **Many-to-Many (N—M)**: Her iki taraftaki kayıtlar birbirleriyle çoklu olarak ilişkilendirilir

İlişkiyi kuran sütuna **foreign key**, hangi tablonun "ana" hangisinin "bağımlı" olduğunu belirleyen yapı ise tasarım kararıdır.

## 7.2 One-to-One (1—1) İlişki

### 7.2.1 Tanım

Bir tablodaki her kayıt, diğer tabloda **en fazla bir** kayıtle eşleşir. FK sütununa **UNIQUE** kısıtı eklenerek tek-tekliği zorunlu kılınır.

Yaygın kullanım amaçları:

- Bir tabloyu fiziksel olarak ikiye bölmek (performans, güvenlik)
- İsteğe bağlı detay verilerini ayrı tutmak (kullanıcı profili)

### 7.2.2 SQL Söz Dizimi

```
-- migrations/01-one-to-one.sql
CREATE TABLE Kullanici (
  Id          INT          PRIMARY KEY,
  Ad          NVARCHAR(100) NOT NULL,
  Email       NVARCHAR(150) UNIQUE NOT NULL,
  SifreHash   NVARCHAR(256)
);

CREATE TABLE KullaniciProfil (
  Id          INT          PRIMARY KEY,
  KullaniciId INT          UNIQUE NOT NULL,  -- UNIQUE → 1-1 zorunlu
  Biyografi   NVARCHAR(500),
  AvatarUrl   NVARCHAR(300),
  DogumTarihi DATE,

  FOREIGN KEY (KullaniciId)
    REFERENCES Kullanici(Id)
    ON DELETE CASCADE
);
```

### 7.2.3 JOIN Sorgusu

```
-- sorgular/kullanici-profil-getir.sql
SELECT k.Ad, k.Email, p.Biyografi
FROM   Kullanici k
LEFT JOIN KullaniciProfil p
  ON k.Id = p.KullaniciId;
```

### 7.2.4 Örnek Veri

**Kullanici** tablosu:

| Id | Ad          | Email           |
|----|-------------|-----------------|
| 1  | Ayşe Yılmaz | ayse@mail.com   |
| 2  | Mehmet Kaya | mehmet@mail.com |

KullaniciProfil tablosu:

| Id | KullaniciId | Biyografi           |
|----|-------------|---------------------|
| 1  | 1           | Yazılım geliştirici |
| 2  | 2           | Veri bilimci        |

KullaniciId=1 ikinci bir profile tekrar kullanılamaz; UNIQUE kısıtı engel olur.

## 7.3 One-to-Many (1–N) İlişki

### 7.3.1 Tanım

Bir tablodaki kayıt, diğer tabloda **birden fazla** kayıtlarla ilişkilendirilebilir. FK sütununa UNIQUE kısıtı **eklenmez** — aynı değer tekrar edebilir.

Bu **en yaygın ilişki türüdür**: kategori-ürün, müşteri-sipariş, yazar-kitap, branş-doktor gibi.

### 7.3.2 SQL Söz Dizimi

```
-- migrations/02-one-to-many.sql
CREATE TABLE Kategori (
  Id INT PRIMARY KEY,
  Ad NVARCHAR(100) NOT NULL,
  Aciklama NVARCHAR(500)
);

CREATE TABLE Urun (
  Id INT PRIMARY KEY,
  KategoriId INT NOT NULL, -- UNIQUE değil
  Ad NVARCHAR(200) NOT NULL,
  Fiyat DECIMAL(10,2),
  Stok INT DEFAULT 0,

  FOREIGN KEY (KategoriId)
    REFERENCES Kategori(Id)
    ON DELETE RESTRICT
);
```

### 7.3.3 JOIN Sorgusu

```
-- sorgular/kategori-urunleri.sql
SELECT u.Ad, u.Fiyat, k.Ad AS Kategori
FROM Urun u
JOIN Kategori k ON u.KategoriId = k.Id
WHERE k.Ad = 'Elektronik';
```

### 7.3.4 Örnek Veri

Kategori:

| Id | Ad         |
|----|------------|
| 1  | Elektronik |
| 2  | Kitap      |

Urun :

| Id | KategoriId | Ad        | Fiyat |
|----|------------|-----------|-------|
| 1  | 1          | Laptop    | 25000 |
| 2  | 1          | Mouse     | 350   |
| 3  | 1          | Klavye    | 750   |
| 4  | 2          | C# Kitabı | 220   |

`KategoriId=1` üç farklı üründe tekrar etmektedir — "1" tarafından "N" tarafına geçişin özüdür.

## 7.4 Many-to-Many (N—M) İlişki

### 7.4.1 Tanım

Her iki taraftaki kayıtlar birbirleriyle çoklu ilişki kurabilir. Bu ilişki **doğrudan kurulamaz** — bir **ara tablo** (junction table / pivot table) zorunludur.

Yaygın kullanım: öğrenci-ders, ürün-etiket, kullanıcı-rol.

### 7.4.2 SQL Söz Dizimi

```
-- migrations/03-many-to-many.sql
CREATE TABLE Ogrenci (
    Id      INT      PRIMARY KEY,
    Ad      NVARCHAR(100) NOT NULL,
    Email   NVARCHAR(150) UNIQUE
);

CREATE TABLE Ders (
    Id      INT      PRIMARY KEY,
    Ad      NVARCHAR(150) NOT NULL,
    Ogretmen NVARCHAR(100),
    Kredi   INT
);

-- Ara tablo
CREATE TABLE OgrenciDers (
    OgrenciId INT      NOT NULL,
    DersId     INT      NOT NULL,
    KayitTarihi DATE     DEFAULT GETDATE(),
    VizeNotu    DECIMAL(5,2),

    -- Composite Primary Key
    PRIMARY KEY (OgrenciId, DersId),

    FOREIGN KEY (OgrenciId)
        REFERENCES Ogrenci(Id)
        ON DELETE CASCADE,
    FOREIGN KEY (DersId)
        REFERENCES Ders(Id)
        ON DELETE CASCADE
);
```

### 7.4.3 JOIN Sorgusu

```
-- sorgular/ogrenci-dersleri.sql
SELECT o.Ad AS Ogrenci, d.Ad AS Ders, od.VizeNotu
FROM   OgrenciDers od
JOIN   Ogrenci o ON od.OgrenciId = o.Id
JOIN   Ders d   ON od.DersId     = d.Id
WHERE  o.Ad = 'Ayşe';
```

#### 7.4.4 Örnek Veri

Ogrenci :

| Id | Ad     |
|----|--------|
| 1  | Ayşe   |
| 2  | Mehmet |
| 3  | Zeynep |

Ders :

| Id | Ad             | Kredi |
|----|----------------|-------|
| 10 | Matematik      | 4     |
| 11 | C# Programlama | 3     |
| 12 | Veritabanı     | 3     |

OgrenciDers (ara tablo):

| OgrenciId | DersId | VizeNotu |
|-----------|--------|----------|
| 1         | 10     | 85       |
| 1         | 11     | 92       |
| 2         | 11     | 78       |
| 3         | 10     | 95       |
| 3         | 12     | 88       |

(OgrenciId, DersId) çifti composite PK olduğu için aynı öğrenci aynı dersi iki kez alamaz.

#### 7.5 İlişki Türü Karşılaştırması

| Özellik                  | 1—1             | 1—N                     | N—M                            |
|--------------------------|-----------------|-------------------------|--------------------------------|
| FK nerede?               | Bağımlı tabloda | "N" tarafındaki tabloda | Ara tabloda iki FK             |
| FK üzerinde UNIQUE ?     | Evet (zorunlu)  | Hayır                   | Composite PK olarak iki birden |
| Ara tablo gerekli mi?    | Hayır           | Hayır                   | Evet                           |
| Doğrudan oluşturulur mu? | Evet            | Evet                    | Hayır — ara tablo şart         |

#### 7.6 Yaygın Senaryolar Tablosu

| Senaryo                     | İlişki türü |
|-----------------------------|-------------|
| Kullanıcı ↔ Pasaport        | 1—1         |
| Yazar → Kitaplar            | 1—N         |
| Öğrenci ↔ Dersler           | N—M         |
| Sipariş → Sipariş Kalemleri | 1—N         |
| Ürün ↔ Etiketler            | N—M         |
| Çalışan → Departman         | N—1         |
| Branş → Doktorlar           | 1—N         |
| Hasta → Randevular          | 1—N         |

## 7.7 Bölüm Özeti

- **1—1**: FK'ya `UNIQUE` kısıtı; tablo bölme amacıyla kullanılır.
- **1—N**: FK "çok" tarafında, `UNIQUE` yoktur; en yaygın ilişki.
- **N—M**: Ara tablo zorunlu; iki FK birlikte composite PK oluşturur.
- İlişkiler `FOREIGN KEY ... REFERENCES ...` ile kurulur.
- N—M ara tabloları ek sütun da içerebilir (kayıt tarihi, not, durum).

# Bölüm 8 — Referans Bütünlüğü ve Cascade Davranışları

## 8.1 Genel Bakış

Bir tabloya foreign key ile bağlı kayıtlar varken ana kayıt silinmek istendiğinde veritabanı ne yapmalı? Bu sorunun cevabı **ON DELETE** kuralıyla belirlenir. Dört seçenek vardır:

| Seçenek                            | Davranış                                            |
|------------------------------------|-----------------------------------------------------|
| <b>CASCADE</b>                     | Alt kayıtlar da otomatik silinir                    |
| <b>RESTRICT</b> / <b>NO ACTION</b> | Alt kayıt varsa silme engellenir, hata fırlatılır   |
| <b>SET NULL</b>                    | Alt tablodaki FK sütunu <b>NULL</b> yapılır         |
| <b>SET DEFAULT</b>                 | Alt tablodaki FK sütunu varsayılan değere ayarlanır |

## 8.2 Söz Dizimi

```
-- migrations/04-foreign-key-davranisi.sql
CREATE TABLE AltTablo (
  Id      INT PRIMARY KEY,
  AnaId   INT,

  FOREIGN KEY (AnaId)
    REFERENCES AnaTablo(Id)
    ON DELETE CASCADE -- veya RESTRICT / SET NULL / SET DEFAULT
    ON UPDATE CASCADE -- güncelleme için ayrı kural
);
```

## 8.3 **CASCADE** Davranışı

### 8.3.1 Tanım

Ana kayıt silindiğinde bağlı alt kayıtlar **otomatik silinir**. Tetikleme bir araya geldiği için "cascade" (basamaklı) adını alır.

### 8.3.2 Kullanım Senaryosu

Alt kayıtların ana kayıt olmadan **anlamı yoksa** uygundur:

- Kullanıcı silinince profili silinmeli
- Sipariş silinince sipariş kalemleri silinmeli
- Anket silinince soruları silinmeli

### 8.3.3 Davranış Örneği

Aşağıdaki yapıda:

```
-- migrations/05-cascade-ornegi.sql
CREATE TABLE Kategori (
  Id INT PRIMARY KEY,
  Ad NVARCHAR(100)
);

CREATE TABLE Urun (
  Id INT PRIMARY KEY,
  KategoriId INT,
  Ad NVARCHAR(200),

  FOREIGN KEY (KategoriId)
    REFERENCES Kategori(Id)
    ON DELETE CASCADE
);
```

Şu komut çalıştırıldığında:

```
DELETE FROM Kategori WHERE Id = 1;
```

Çıktı: (yok — yan etki: `Kategori.Id=1` silinir; `Urun` tablosunda `KategoriId=1` olan tüm kayıtlar da otomatik silinir)

## 8.4 `RESTRICT` / `NO ACTION` Davranışı

### 8.4.1 Tanım

Alt tabloda bağlı kayıt varken ana kaydı silmeyi **engeller**, FK constraint hatası fırlatır.

### 8.4.2 Kullanım Senaryosu

Alt kayıtların korunması gerektiğinde (yanlışlıkla silme riski):

- Yazar silinmek istendiğinde kitaplar korunmalı
- Müşteri silinmek istendiğinde geçmiş siparişler korunmalı

### 8.4.3 Davranış Örneği

```
-- ON DELETE RESTRICT tanımlı varsayalım
DELETE FROM Kategori WHERE Id = 1;
```

Çıktı (hata):

```
The DELETE statement conflicted with the REFERENCE constraint
"FK_Urun_Kategori". The conflict occurred in database "HastaneDB",
table "dbo.Urun", column 'KategoriId'.
The statement has been terminated.
```

Neden hata: `Urun` tablosunda `KategoriId=1` olan kayıtlar var; ana kayıt silinemez.

## 8.5 `SET NULL` Davranışı

### 8.5.1 Tanım

Ana kayıt silindiğinde, alt tablodaki FK sütunu `NULL` değerine ayarlanır. FK sütunu `NULL` 'a izin vermeli (`NOT NULL` olmamalı).

### 8.5.2 Kullanım Senaryosu

Alt kayıtların yaşamaya devam etmesi gerektiğinde ama bağlantısının kopmasında sakınca olmadığında:

- Çalışan silinince ona atanan görevler "atanmamış" olur
- Yazar silinince kitap "anonim" hale gelir

### 8.5.3 Davranış Örneği

```
DELETE FROM Kategori WHERE Id = 1;
```

Çıktı: (yok — yan etki: `Kategori.Id=1` silinir; `Urun` tablosunda `KategoriId=1` olan tüm kayıtların `KategoriId` sütunu `NULL` olur)

### 8.5.4 Önemli Kısıt

N—M ara tablolarında `SET NULL` **kullanılamaz**. Ara tabloda FK sütunu Composite Primary Key'in parçası olduğu için `NULL` değer alamaz; veritabanı hata fırlatır.

## 8.6 İlişki Türü ↔ Davranış Tablosu

| İlişki            | Senaryo                     | Önerilen davranış |
|-------------------|-----------------------------|-------------------|
| 1—1               | Kullanıcı → Profil          | CASCADE           |
| 1—N (bağımlı)     | Sipariş → Sipariş Kalemleri | CASCADE           |
| 1—N (bağımsız)    | Kategori → Ürünler          | RESTRICT          |
| 1—N (geçmiş veri) | Çalışan → Görevler          | SET NULL          |
| N—M (her iki FK)  | Öğrenci ↔ Ders              | CASCADE           |

## 8.7 Zincirleme CASCADE Riski

CASCADE'in tetikleyebileceği zincirleme silme uzunsa beklenmedik veri kaybı oluşabilir. Aşağıdaki senaryoda:

```
Kategori (silinen)
├─ CASCADE → Urun
│   └─ CASCADE → StokHareketi
│       └─ CASCADE → StokLogu
```

Tek bir `DELETE FROM Kategori WHERE Id=1` komutu **dört tabloda** silme tetikler. Tasarımda bu zincirin uzunluğu kontrol altında tutulmalıdır.

## 8.8 Tam Tasarım Örneği — Sipariş Sistemi

### Örnek 8.8.1 — Karma davranış stratejisi

```
-- migrations/06-tam-tasarim.sql
CREATE TABLE Kullanici (
    Id INT PRIMARY KEY,
    Ad NVARCHAR(100)
);

CREATE TABLE KullaniciProfil (
    Id INT PRIMARY KEY,
    KullaniciId INT UNIQUE NOT NULL,
    Biyografi NVARCHAR(500),

    FOREIGN KEY (KullaniciId)
        REFERENCES Kullanici(Id)
        ON DELETE CASCADE -- 1-1, birlikte gider
);

CREATE TABLE Siparis (
    Id INT PRIMARY KEY,
    KullaniciId INT,
    Tutar DECIMAL(10,2),

    FOREIGN KEY (KullaniciId)
        REFERENCES Kullanici(Id)
        ON DELETE SET NULL -- geçmiş korunsun, sahip kalsın
);

CREATE TABLE SiparisKalem (
    Id INT PRIMARY KEY,
    SiparisId INT NOT NULL,
    UrunId INT NOT NULL,
    Adet INT,

    FOREIGN KEY (SiparisId)
        REFERENCES Siparis(Id)
        ON DELETE CASCADE, -- sipariş gidince kalem de
    FOREIGN KEY (UrunId)
        REFERENCES Urun(Id)
        ON DELETE RESTRICT -- ürün silinmesin
);
```

Bu tasarım üç farklı stratejiyi birlikte kullanır: profili `CASCADE`, sipariş ilişkisini `SET NULL`, ürün ilişkisini `RESTRICT`.

## 8.9 Yaygın Hatalar ve Çözümleri

**Hata: N—M ara tabloda** `SET NULL`

```
-- Hatalı
CREATE TABLE OgrenciDers (
    OgrenciId INT,
    DersId INT,
    PRIMARY KEY (OgrenciId, DersId),

    FOREIGN KEY (OgrenciId)
        REFERENCES Ogrenci(Id)
        ON DELETE SET NULL -- hata!
);
```

Çıktı:

```
Cannot create FOREIGN KEY constraint with ON DELETE SET NULL
because the column 'OgrenciId' is part of the PRIMARY KEY.
```

Neden hata: PK parçası olan sütun `NULL` olamaz. `CASCADE` veya `RESTRICT` kullanılmalıdır.

## 8.10 Bölüm Özeti

---

- `CASCADE` : alt kayıtlar da silinir — bağımlı veriler için.
  - `RESTRICT` : alt kayıt varsa silme engellenir — koruma için.
  - `SET NULL` : FK sütunu `NULL` olur — bağımsız yaşayan alt kayıtlar için.
  - N—M ara tablolarında `SET NULL` kullanılamaz; tercih `CASCADE` 'dir.
  - Tasarımda her FK için davranış bilinçli seçilmeli; tek varsayılan yoktur.
  - Uzun CASCADE zincirleri beklenmedik veri kaybı riski taşır.
-

# Bölüm 9 — Razor View Engine

## 9.1 Genel Bakış

Razor, ASP.NET Core MVC'nin **view motoru**dur. Razor dosyaları (`.cshtml` uzantılı) HTML ile C# kodunu aynı dosyada birleştirir. Sunucu tarafında işlenir, tarayıcıya saf HTML çıktısı gönderilir.

İki temel yapı taşı vardır:

- **HTML** — sayfanın iskeleti
- **@ ile başlayan C# kodu** — dinamik içerik üretme

## 9.2 @ Operatörü ve Razor Söz Dizimi

### 9.2.1 Temel Yapılar

| Söz dizimi             | İşlevi                                               |
|------------------------|------------------------------------------------------|
| @model Tip             | View'ın beklediği model tipini bildirir              |
| @using AdAlanı         | C# namespace ekler                                   |
| @{ ... }               | Çok satırlı C# bloğu                                 |
| @degisken              | Bir değişkenin değerini HTML'e yazar                 |
| @Model.Alan            | Modelin bir alanına erişir                           |
| @if (...) { ... }      | Koşullu HTML üretimi                                 |
| @foreach (...) { ... } | Döngüyle HTML üretimi                                |
| @* yorum *@            | Razor yorumu (HTML'e çıkmaz)                         |
| @section Ad { ... }    | Layout'taki belirli bölgeye içerik gönderir          |
| @:                     | Tek satır düz metin (Razor parser'ı geçici durdurur) |

### 9.2.2 Sayfa İskeleti

```
@* Views/Hasta/Ekle.cshtml *@
@using HastaneMVC.ViewModels
@model RandevuEkleVM

@{
    ViewData["Title"] = "Yeni Randevu Oluştur";
    var bugun = DateTime.Today.ToString("dd.MM.yyyy");
}

<h2>Randevu Formu</h2>
<p>Bugünün tarihi: @bugun</p>
<p>Hasta sayısı: @Model.HastaListesi.Count</p>

@if (Model.HastaListesi.Any())
{
    <ul>
        @foreach (var h in Model.HastaListesi)
        {
            <li>@h.AdSoyad</li>
        }
    </ul>
}
else
{
    <p>Henüz hasta yok.</p>
}

@section Scripts {
    <script>
        console.log("Sayfa yüklendi");
    </script>
}
```

Çıktı: (yok — yan etki: sayfada başlık, tarih, hasta sayısı ve liste render edilir)

### 9.2.3 Sıra Bağımlılığı

Sayfanın **en üstünde** şu sıraya uyulur:

1. `@using` direktifleri (varsa)
2. `@model` direktifi
3. `@{ ... }` bloğu (ViewData, vb.)
4. HTML içerik
5. `@section` blokları (en alta)

## 9.3 Layout, ViewStart ve ViewImports

### 9.3.1 Layout

Tüm sayfalarda paylaşılan iskelet (üst menü, footer, CSS/JS referansları) ortak bir `_Layout.cshtml` dosyasında tanımlanır.

```

@* Views/Shared/_Layout.cshtml *@
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="utf-8" />
  <title>@ViewData["Title"] - HastaneMVC</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
</head>
<body>
  <nav class="navbar navbar-light bg-light">
    <a class="navbar-brand" asp-controller="Home" asp-action="Index">HastaneMVC</a>
  </nav>

  <main class="container">
    @RenderBody()          @* sayfanın HTML içeriği buraya gelir *@
  </main>

  <script src="~/lib/jquery/dist/jquery.min.js"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

### 9.3.2 \_ViewStart.cshtml

Tüm view'lar için varsayılan layout'u belirler.

```

@* Views/_ViewStart.cshtml *@
@{
  Layout = "_Layout";
}

```

### 9.3.3 \_ViewImports.cshtml

Tüm view'larda paylaşılan `@using` ve Tag Helper kayıtları.

```

@* Views/_ViewImports.cshtml *@
@using HastaneMVC
@using HastaneMVC.Models
@using HastaneMVC.ViewModels
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

```

`@addTagHelper` satırı **olmadan** Tag Helper'lar çalışmaz.

## 9.4 @section Scripts Kullanımı

Layout'ta tanımlı bir bölgeye sayfaya özel JS gönderilir. jQuery body sonuna yüklendiği için sayfa scriptleri layout'un sonunda toplanır.

### Örnek 9.4.1 — Sayfaya özel script bölümü

```

@* Views/Randevu/Ekle.cshtml *@
@section Scripts {
  @* Validation kütüphanesini dahil et *@
  @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }

  <script>
    $(document).ready(function () {
      console.log("Randevu sayfası hazır");
    });
  </script>
}

```

Çıktı: (yok — yan etki: layout'taki `@await RenderSectionAsync("Scripts")` çağrısı bu içeriği body sonuna yerleştirir;

validation kütüphanesi ve sayfa scripti yüklenir)

## 9.5 Bölüm Özeti

---

- `.cshtml` dosyaları HTML + C# (Razor söz dizimi) birleşimidir.
  - `@model` direktifi sayfanın beklediği veri tipini bildirir; `@Model` ile erişilir.
  - `@if`, `@foreach`, `@{ }` ile dinamik içerik üretilir.
  - `_Layout.cshtml` ortak iskeleti, `_ViewStart.cshtml` varsayılan layout'u, `_ViewImports.cshtml` paylaşılan namespace'leri tanımlar.
  - `@section Scripts` sayfaya özel JS'i layout'taki yere yerleştirir.
  - Tag Helper kullanmak için `_ViewImports.cshtml` 'de `@addTagHelper` satırı zorunludur.
-

# Bölüm 10 — Tag Helpers

## 10.1 Genel Bakış

Tag Helper, Razor view'larında HTML etiketlerine `asp-` önekli özellikler ekleyerek MVC'nin otomatik olarak doğru `id`, `name`, `href`, `action` gibi attribute'ları üretmesini sağlayan yardımcı yapıdır.

Tag Helper'ların üç ana faydası:

- Model bağlama otomasyonu** — `name` attribute'u modelle eşleştirilir
- URL üretimi** — controller/action adlarından otomatik URL
- Tip güvenliği** — derleme zamanında property kontrolü

## 10.2 Söz Dizimi Genel Kuralı

Bir HTML etiketine `asp-` ile başlayan bir özellik eklediğinizde Razor o özelliği işler ve karşılığı olan HTML attribute'unu üretir.

### 10.2.1 Nokta → Alt Çizgi Kuralı

`asp-for="YeniRandevu.HastaId"` yazıldığında üretilen attribute'lar:

| HTML Attribute     | Değer                            | Kural                                                                                                                                    |
|--------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>    | <code>YeniRandevu_HastaId</code> | Nokta alt çizgiye dönüşür                                                                                                                |
| <code>name</code>  | <code>YeniRandevu.HastaId</code> | Nokta korunur (model binding için)                                                                                                       |
| <code>value</code> | Modeldeki mevcut değer           | Otomatik doldurulur                                                                                                                      |
| <code>type</code>  | Property tipine göre             | <code>int</code> → <code>number</code> , <code>DateTime</code> → <code>datetime-local</code> , <code>bool</code> → <code>checkbox</code> |

## 10.3 `asp-for` — Veri Bağlama

### 10.3.1 Tanım

Bir HTML etiketini model property'siyle bağlar. `<label>`, `<input>`, `<textarea>`, `<select>` etiketlerinde kullanılır.

### 10.3.2 `<label>` ile Kullanım

#### Örnek 10.3.1

```
@* Views/Randevu/Ekle.cshtml - label *@
<label asp-for="YeniRandevu.HastaId" class="form-label fw-bold">
    Randevu Alacak Hasta
</label>
```

Üretilen HTML:

```
<label for="YeniRandevu_HastaId" class="form-label fw-bold">
    Randevu Alacak Hasta
</label>
```

Çıktı: (yok — yan etki: HTML akışına label etiketi eklenir, `for` attribute'u ilgili input'a bağlanır)

### 10.3.3 `<input>` ile Kullanım

#### Örnek 10.3.2

```
@* Views/Randevu/Ekle.cshtml - datetime input *@
<input asp-for="YeniRandevu.TarihSaat"
       type="datetime-local"
       class="form-control form-control-lg"
       required />
```

Üretilen HTML (mevcut değer `2025-06-15T10:30` ise):

```
<input type="datetime-local"
       id="YeniRandevu_TarihSaat"
       name="YeniRandevu.TarihSaat"
       value="2025-06-15T10:30"
       class="form-control form-control-lg"
       required />
```

### 10.3.4 Hidden Field

#### Örnek 10.3.3 — ID'yi gizli taşımak

```
@* Views/Randevu/Guncelle.cshtml *@
<input type="hidden" asp-for="YeniRandevu.Id" />
```

Üretilen HTML:

```
<input type="hidden" id="YeniRandevu_Id" name="YeniRandevu.Id" value="42" />
```

Bu, **güncelleme formlarında kritiktir**. ID gizli taşınmazsa POST'ta `Id = 0` gelir; veritabanı yeni kayıt ekler, güncelleme yapmaz.

### 10.3.5 `<textarea>` ile Kullanım

#### Örnek 10.3.4

```
@* Views/Randevu/Guncelle.cshtml *@
<textarea asp-for="YeniRandevu.DoktorNotu"
          class="form-control"
          rows="4"
          placeholder="Triyaj notu..."></textarea>
```

Üretilen HTML (mevcut değer `"Triyaj: Orta"` ise):

```
<textarea id="YeniRandevu_DoktorNotu"
          name="YeniRandevu.DoktorNotu"
          class="form-control"
          rows="4"
          placeholder="Triyaj notu...">Triyaj: Orta</textarea>
```

## 10.4 `<form>` Tag Helper — `asp-action` ve `asp-controller`

### 10.4.1 Tanım

Bir formun POST/GET edeceği URL'i otomatik üretir.

### 10.4.2 Aynı Controller İçinde Action

```
@* Views/Randevu/Ekle.cshtml *@
<form asp-action="Ekle" method="post">
    ...
</form>
```

Üretilen HTML (sayfa `RandevuController` altında ise):

```
<form action="/Randevu/Ekle" method="post">
    ...
</form>
```

### 10.4.3 Farklı Controller'a Yönlendirme

```
<form asp-action="Kaydet" asp-controller="Hasta" method="post">
    ...
</form>
```

Üretilen HTML:

```
<form action="/Hasta/Kaydet" method="post">
    ...
</form>
```

## 10.5 <select> Tag Helper — asp-for + asp-items

### 10.5.1 Tanım

`asp-for` → seçilen değerin hangi property'ye gideceğini, mevcut değerin hangisi olduğunu belirtir.

`asp-items` → option listesini bir `SelectList` nesnesinden doldurur.

### 10.5.2 SelectList Yapımı

```
new SelectList(kaynakListe, "valueAlani", "textAlani")
```

| Parametre                 | Anlamı                                                            |
|---------------------------|-------------------------------------------------------------------|
| <code>kaynakListe</code>  | <code>IEnumerable&lt;T&gt;</code> — option'ları üreten koleksiyon |
| <code>"valueAlani"</code> | Her option'ın <code>value</code> attribute'una giren property     |
| <code>"textAlani"</code>  | Her option'ın kullanıcıya görünen metni                           |

### 10.5.3 Söz Dizimi

#### Örnek 10.5.1 — Hasta seçim listesi

```
@* Views/Randevu/Ekle.cshtml *@
<select asp-for="YeniRandevu.HastaId"
        asp-items="@((new SelectList(Model.HastaListesi, "Id", "AdSoyad")))"
        class="form-select form-select-lg"
        required>
    <option value="">-- Lütfen Hasta Seçiniz --</option>
</select>
```

Üretilen HTML ( `HastaListesi` 'nde üç hasta varsa ve mevcut `HastaId = 2` ise):

```
<select id="YeniRandevu_HastaId"
        name="YeniRandevu.HastaId"
        class="form-select form-select-lg"
        required>
    <option value="">-- Lütfen Hasta Seçiniz --</option>
    <option value="1">Ali Veli</option>
    <option value="2" selected>Ayşe Yılmaz</option>
    <option value="3">Mehmet Demir</option>
</select>
```

Mevcut değer için `selected` attribute'u otomatik eklenir — bu özellik **güncelleme formlarında özellikle önemlidir**.

### 10.5.4 `asp-items` Olmadan, Sadece jQuery için

Bir `<select>` form gönderiminde POST'a katılmayacaksa (yalnızca jQuery ile tetikleyici olarak kullanılacaksa) `asp-for` yazılmaz:

#### Örnek 10.5.2 — Branş seçimi (POST'a bağlı değil)

```
@* Views/Randevu/Ekle.cshtml *@
<select id="bransSelect"
    asp-items="@((new SelectList(Model.BransListesi, "Id", "BransAdi")))"
    class="form-select form-select-lg">
    <option value="">-- Önce Branş Seçiniz --</option>
</select>
```

`asp-for` olmadığı için `name` attribute'u üretilmez — bu select'in değeri form POST edildiğinde sunucuya gönderilmez.

## 10.6 `<a>` Tag Helper — Link Üretimi

### 10.6.1 Tanım

`<a>` etiketinde `asp-action`, `asp-controller`, `asp-route-{paramAdi}` kullanılarak `href` URL'i otomatik üretilir.

### 10.6.2 Söz Dizimi

#### Örnek 10.6.1 — Basit link

```
<a asp-action="Index" class="btn btn-outline-secondary">
    Listeye Dön
</a>
```

Üretilen HTML:

```
<a href="/Randevu/Index" class="btn btn-outline-secondary">
    Listeye Dön
</a>
```

#### Örnek 10.6.2 — Route parametresi ile

```
<a asp-action="Detay" asp-route-id="@item.Id" class="btn btn-info">
    Detay
</a>
```

Üretilen HTML (`item.Id = 5` ise):

```
<a href="/Randevu/Detay/5" class="btn btn-info">
    Detay
</a>
```

#### Örnek 10.6.3 — Çoklu parametre ve farklı controller

```
<a asp-controller="Randevu"
    asp-action="Filtrele"
    asp-route-kategori="Dahiliye"
    asp-route-sayfa="2">
    Sonraki sayfa
</a>
```

Üretilen HTML:

```
<a href="/Randevu/Filtrele?kategori=Dahiliye&sayfa=2">
    Sonraki sayfa
</a>
```

## 10.7 Validation Tag Helper'ları

### 10.7.1 `asp-validation-for`

Belirli bir property için hata mesajını gösterir.

```
<span asp-validation-for="YeniRandevu.HastaId" class="text-danger"></span>
```

Üretilen HTML (hata yoksa):

```
<span class="text-danger field-validation-valid" data-valmsg-for="YeniRandevu.HastaId" data-valmsg-replace="true"></span>
```

Üretilen HTML (hata varsa):

```
<span class="text-danger field-validation-error" data-valmsg-for="YeniRandevu.HastaId">
    Hasta alanı zorunludur.
</span>
```

### 10.7.2 `asp-validation-summary`

Tüm hataları tek bir blokta özetler.

```
<div asp-validation-summary="ModelOnly" class="alert alert-danger"></div>
```

| Değer     | Anlamı                                     |
|-----------|--------------------------------------------|
| None      | Hiç gösterme                               |
| ModelOnly | Property'lere bağlı olmayan model hataları |
| All       | Tüm hatalar (property bazlı + model bazlı) |

## 10.8 Etiket—Tag Helper Eşleme Tablosu

| HTML Etiketi | Kullanılan Tag Helper                                                                | Üretilen Attribute(lar)                                                       |
|--------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <form>       | <code>asp-action</code> , <code>asp-controller</code>                                | <code>action</code>                                                           |
| <a>          | <code>asp-action</code> , <code>asp-controller</code> , <code>asp-route- {ad}</code> | <code>href</code>                                                             |
| <label>      | <code>asp-for</code>                                                                 | <code>for</code>                                                              |
| <input>      | <code>asp-for</code>                                                                 | <code>id</code> , <code>name</code> , <code>type</code> , <code>value</code>  |
| <textarea>   | <code>asp-for</code>                                                                 | <code>id</code> , <code>name</code> , içerik                                  |
| <select>     | <code>asp-for</code> , <code>asp-items</code>                                        | <code>id</code> , <code>name</code> , <option> listesi, <code>selected</code> |
| <span>       | <code>asp-validation-for</code>                                                      | hata mesajı içeriği                                                           |
| <div>        | <code>asp-validation-summary</code>                                                  | tüm hataların listesi                                                         |

## 10.9 Bölüm Özeti

- Tag Helper'lar `asp-` önekli attribute'larla HTML üretimini sade ve güvenli yapar.
- `asp-for` modelle bağlama: label, input, textarea, select için.
- `asp-action` / `asp-controller` : form ve link için URL üretimi.
- `asp-items` + `SelectList` : select option'larını otomatik oluşturma.
- `asp-route- {paramAdi}` : link/form URL'ine parametre ekleme.
- `asp-validation-for` / `asp-validation-summary` : client-side hata gösterimi.

- Tag Helper kullanmak için `_ViewImports.cshtml` 'de `@addTagHelper` satırı şarttır.
-

# Bölüm 11 — Form İşleme ve Model Binding

## 11.1 Genel Bakış

Form **HTTP POST** mekanizmasıyla sunucuya veri gönderir. ASP.NET Core MVC'nin **model binding** sistemi, form alanlarındaki `name` attribute'larını okuyup karşılık gelen action parametresinin property'lerine otomatik eşler.

Bu eşleme `name` attribute'una göre yapılır; bu yüzden Tag Helper kullanımı kritiktir — Tag Helper doğru `name` 'i otomatik üretir.

## 11.2 Form Gönderiminin Tam Akışı

```
sequenceDiagram
    participant T as Tarayıcı
    participant K as Controller
    participant V as View
    participant DB as Veritabanı

    T->>K: GET /Randevu/Ekle
    K->>V: View(vm) — boş form
    V->>T: HTML (form + listeler)
    T->>T: Kullanıcı doldurur
    T->>K: POST /Randevu/Ekle (form data)
    K->>K: Model Binding (name → property)
    K->>K: ModelState.IsValid?
    alt Geçerli
        K->>DB: SaveChanges()
        K->>T: Redirect /Randevu/Index
    else Geçersiz
        K->>V: View(vm) — hatalı haliyle
        V->>T: HTML + hata mesajları
    end
end
```

## 11.3 GET / POST Action Çifti

### 11.3.1 GET — Formu Göster

```
// Controllers/RandevuController.cs
public IActionResult Ekle()
{
    var vm = new RandevuEkleVM
    {
        YeniRandevu = new Randevu(),
        HastaListesi = _context.Hastalar.ToList(),
        BransListesi = _context.Branslar.ToList(),
        DoktorListesi = new List<Doktor>() // başta boş
    };
    return View(vm);
}
```

### 11.3.2 POST — Formu Kaydet

```
// Controllers/RandevuController.cs
[HttpPost]
public IActionResult Ekle(RandevuEkleVM vm)
{
    if (!ModelState.IsValid)
    {
        // Hata varsa listeleri yeniden yükle (boş gelir aksi halde)
        vm.HastaListesi = _context.Hastalar.ToList();
        vm.BransListesi = _context.Branslar.ToList();
        return View(vm);
    }

    _context.Randevular.Add(vm.YeniRandevu);
    _context.SaveChanges();
    return RedirectToAction("Index");
}
```

## 11.4 Model Binding Detayı

### 11.4.1 İsim Eşleştirme

Tarayıcı POST gövdesi şuna benzer:

```
YeniRandevu.HastaId=3&YeniRandevu.DoktorId=7&YeniRandevu.TarihSaat=2025-06-15T10:30
```

MVC bunu okur ve `RandevuEkleVM` parametresinin alanlarına eşler:

| Form name             | Hangi property'ye gider  |
|-----------------------|--------------------------|
| YeniRandevu.HastaId   | vm.YeniRandevu.HastaId   |
| YeniRandevu.DoktorId  | vm.YeniRandevu.DoktorId  |
| YeniRandevu.TarihSaat | vm.YeniRandevu.TarihSaat |

Bu eşleme `asp-for` Tag Helper'ı sayesinde otomatiktir; elle `name="YeniRandevu.HastaId"` yazılması gerekmez.

### 11.4.2 Tip Dönüşümü

MVC, string form verisini hedef property tipine otomatik çevirir:

| Property tipi | Form değeri        | Sonuç                          |
|---------------|--------------------|--------------------------------|
| int           | "42"               | 42                             |
| int           | "abc"              | 0 + ModelState hatası          |
| DateTime      | "2025-06-15T10:30" | DateTime (2025, 6, 15, 10, 30) |
| bool          | "true" / "on"      | true                           |
| decimal       | "9.99"             | 9.99m                          |

## 11.5 ViewModel Tasarımı

### 11.5.1 Tanım

ViewModel, bir view'ın ihtiyaç duyduğu **tüm verileri** (form verisi + listeler + ek bilgiler) bir araya getiren sınıftır. Entity'ler doğrudan view'a gönderilmez — ViewModel ile sarmalanır.

### 11.5.2 RandevuEkleVM Örneği

```
// ViewModels/RandevuEkleVM.cs
public class RandevuEkleVM
{
    // 1) Form verisi
    public Randevu YeniRandevu { get; set; } = new();

    // 2) Açılır liste verileri
    public List<Hasta> HastaListesi { get; set; } = new();
    public List<Brans> BransListesi { get; set; } = new();
    public List<Doktor> DoktorListesi { get; set; } = new();
}
```

### 11.5.3 Entity vs ViewModel

| Entity (Randevu)                 | ViewModel (RandevuEkleVM)                    |
|----------------------------------|----------------------------------------------|
| Veritabanı tablo karşılığı       | View'ın görüntülemesi için hazırlanmış paket |
| Yalnızca veri alanları           | Veri + listeler + UI'ye özel ek alanlar      |
| EF Core ile veritabanına yazılır | Asla doğrudan veritabanına gitmez            |

## 11.6 Validation — Veri Doğrulama

### 11.6.1 Model'de Data Annotation

```
// Models/Hasta.cs
public class Hasta
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Ad alanı zorunludur")]
    [StringLength(100, ErrorMessage = "En fazla 100 karakter")]
    public string Adi { get; set; }

    [Required(ErrorMessage = "TC zorunludur")]
    [StringLength(11, MinimumLength = 11, ErrorMessage = "TC 11 hane olmalıdır")]
    public string TcKimlik { get; set; }

    [Range(0, 150, ErrorMessage = "Yaş 0-150 arası olmalıdır")]
    public int Yas { get; set; }

    [EmailAddress(ErrorMessage = "Geçerli e-posta giriniz")]
    public string Eposta { get; set; }
}
```

### 11.6.2 Yaygın Annotation'lar

| Attribute             | Açıklama                                     |
|-----------------------|----------------------------------------------|
| [Required]            | Zorunlu alan                                 |
| [StringLength(max)]   | Maksimum karakter sayısı                     |
| [Range(min, max)]     | Sayısal aralık                               |
| [EmailAddress]        | E-posta formatı                              |
| [Phone]               | Telefon formatı                              |
| [RegularExpression]   | Regex kalıbı                                 |
| [Compare("Diger")]    | Başka property ile aynı olmalı (şifre onayı) |
| [Display(Name="...")] | Label'larda gösterilecek isim                |

### 11.6.3 Controller'da Kontrol

```
[HttpPost]
public IActionResult Ekle(RandevuEkleVM vm)
{
    if (!ModelState.IsValid) // tüm validation kuralları kontrol edilir
    {
        // Aynı sayfayı yeniden göster, hatalar gösterilsin
        vm.HastaListesi = _context.Hastalar.ToList();
        return View(vm);
    }

    _context.Randevular.Add(vm.YeniRandevu);
    _context.SaveChanges();
    return RedirectToAction("Index");
}
```

### 11.6.4 View'da Hata Mesajları

```
@* Views/Randevu/Ekle.cshtml *@
<div class="mb-4">
    <label asp-for="YeniRandevu.HastaId" class="form-label fw-bold">Hasta</label>
    <select asp-for="YeniRandevu.HastaId"
        asp-items="@new SelectList(Model.HastaListesi, "Id", "AdSoyad")"
        class="form-select" required>
        <option value="">-- Seçiniz --</option>
    </select>
    <span asp-validation-for="YeniRandevu.HastaId" class="text-danger"></span>
</div>
```

### 11.6.5 Client-Side Validation

JavaScript tarafında da validation çalışması için `_ValidationScriptsPartial` 'in dahil edilmesi gerekir.

```
@* Views/Randevu/Ekle.cshtml *@
@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}
```

Bu partial jQuery Validation ve unobtrusive validation kütüphanelerini yükler — kullanıcı submit etmeden önce alan doğrulanır.

## 11.7 Action Sonuçları

Controller action'ları çeşitli sonuç tipleri döndürür:

| Dönüş tipi                              | Anlamı                                |
|-----------------------------------------|---------------------------------------|
| <code>View()</code>                     | Aynı isimde view'ı render eder        |
| <code>View(model)</code>                | Belirtilen modelle view'ı render eder |
| <code>View("AdiBelirtilmis")</code>     | Farklı isimde view render eder        |
| <code>RedirectToAction("X")</code>      | HTTP 302 — X action'ına yönlendirir   |
| <code>RedirectToAction("X", "Y")</code> | X action, Y controller'a yönlendirir  |
| <code>NotFound()</code>                 | HTTP 404                              |
| <code>BadRequest()</code>               | HTTP 400                              |
| <code>Json(nesne)</code>                | JSON yanıt (AJAX için)                |
| <code>Content("metin")</code>           | Düz metin yanıt                       |

## 11.8 POST-Redirect-GET Kalıbı

Başarılı POST sonrasında doğrudan view dönmek yerine **redirect** yapmak standart yaklaşımdır:

```
[HttpPost]
public IActionResult Ekle(RandevuEkleVM vm)
{
    if (!ModelState.IsValid) return View(vm);

    _context.Randevular.Add(vm.YeniRandevu);
    _context.SaveChanges();

    return RedirectToAction("Index"); // POST-Redirect-GET
}
```

Bu kalıbın yararı: kullanıcı sayfayı yenilediğinde formun tekrar gönderilmesini engeller. Tarayıcı, redirect sonrasındaki GET isteğini yeniler.

## 11.9 Bölüm Özeti

- Form POST'unda `name` attribute'u model property yoluyla birebir eşleşir.
- Tag Helper'lar (`asp-for`) bu `name` 'i otomatik üretir — elle yazmak hatalara açıktır.
- ViewModel pattern: entity'leri doğrudan view'a göndermek yerine ihtiyaç duyulan tüm veriyi tek pakette sunar.
- Validation iki katmanda yapılır: model üzerindeki Data Annotation + Controller'daki `ModelState.IsValid` kontrolü.
- Client-side validation için `_ValidationScriptsPartial` dahil edilir.
- Başarılı POST sonrası `RedirectToAction` ile yönlendirilir (POST-Redirect-GET).

## Bölüm 12 — AJAX ile Asenkron Veri Akışı

### 12.1 Genel Bakış

AJAX (Asynchronous JavaScript And XML), tarayıcının sayfayı **yenilemeden** sunucuya istek gönderip yanıt aldığı tekniktir. Modern web uygulamalarında JSON formatı XML yerine geçmiştir.

ASP.NET Core MVC + jQuery kombinasyonunda tipik kullanım: bir form alanı değişince **sunucudan ilgili veriyi çekip başka bir alanı doldurmak** (bağımlı dropdown, zincirleme listeler).

### 12.2 jQuery `$.ajax()` Söz Dizimi

#### 12.2.1 Temel Yapı

```
// Views/Randevu/Ekle.cshtml — script bloğu
$.ajax({
  url:      '/Controller/Action',    // istek URL'i
  type:     'GET',                  // HTTP metodu (GET, POST, PUT, DELETE)
  data:     { param1: deger1 },      // sunucuya gönderilen veri
  success: function (gelenVeri) {   // başarılı yanıt callback'i
    // gelenVeri: sunucudan dönen JSON
  },
  error:    function (xhr, status) { // hata callback'i
    // bağlantı koptu, 4xx/5xx hata, vb.
  }
});
```

#### 12.2.2 Parametreler

| Parametre             | Anlamı                                                                                                       |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| <code>url</code>      | İstek yapılacak adres                                                                                        |
| <code>type</code>     | HTTP metodu: <code>GET</code> (okuma), <code>POST</code> (yazma), <code>PUT</code> , <code>DELETE</code>     |
| <code>data</code>     | Sunucuya gönderilecek veri (GET'te query string, POST'ta body)                                               |
| <code>success</code>  | HTTP 2xx yanıt geldiğinde çağrılır; gelen veri parametre olur                                                |
| <code>error</code>    | Ağ hatası, 4xx, 5xx gibi başarısız durumlarda çağrılır                                                       |
| <code>dataType</code> | Beklenen yanıt tipi ( <code>'json'</code> , <code>'html'</code> , <code>'text'</code> ) — otomatik algılanır |

### 12.3 Controller Endpoint'i

#### 12.3.1 JSON Döndüren Action

```
// Controllers/RandevuController.cs
public IActionResult DoktorlariGetir(int bransId)
{
    var doktorlar = _context.Doktorlar
        .Where(d => d.BransId == bransId)
        .Select(d => new
        {
            d.Id,
            d.AdSoyad
        })
        .ToList();

    return Json(doktorlar);
}
```

### 12.3.2 Üretilen Yanıt

```
[
  { "id": 1, "adSoyad": "Dr. Ahmet Kaya" },
  { "id": 2, "adSoyad": "Dr. Merve Çelik" }
]
```

Not: JSON serileştirmede C# property adlarının ilk harfi **küçük harfe çevrilir** (camelCase). JavaScript tarafında `doktor.id`, `doktor.adSoyad` olarak erişilir.

## 12.4 Tam Akış — Branş Seçince Doktor Listesi

### 12.4.1 Akış Diyagramı

```
sequenceDiagram
    participant K as Kullanıcı
    participant T as Tarayıcı (jQuery)
    participant S as Sunucu (Controller)
    participant DB as Veritabanı

    K->>T: Branş seçer
    T->>T: change olayı tetiklenir
    T->>S: GET /Randevu/DoktorlariGetir?bransId=2
    S->>DB: SELECT * FROM Doktorlar WHERE BransId=2
    DB-->>S: Doktor listesi
    S-->>T: JSON: [{id:1, adSoyad:"Dr. A"}, ...]
    T->>T: success callback — option'ları doldur
    T->>K: Doktor select aktif, dolu
```

### 12.4.2 Tam JavaScript Kodu

```
// Views/Randevu/Ekle.cshtml — @section Scripts
$(document).ready(function () {
    $('#bransSelect').change(BransDegisti);
});

function BransDegisti() {
    var secilenBransId = $('#bransSelect').val();

    if (secilenBransId !== "") {
        AjaxIleDoktorlariGetir(secilenBransId);
    } else {
        DoktorKutusunuKilitle();
    }
}

function AjaxIleDoktorlariGetir(bransId) {
    $.ajax({
        url:    '/Randevu/DoktorlariGetir',
        type:   'GET',
        data:   { bransId: bransId },
        success: DoktorKutusunuDoldur,
        error:   SunucuHatasi
    });
}

function DoktorKutusunuDoldur(gelenVeri) {
    var doktorSelect = $('#doktorSelect');
    doktorSelect.empty();
    doktorSelect.append('<option value="">-- Lütfen Doktor Seçiniz --</option>');

    for (var i = 0; i < gelenVeri.length; i++) {
        var d = gelenVeri[i];
        doktorSelect.append(
            '<option value="' + d.id + '">' + d.adSoyad + '</option>'
        );
    }
    doktorSelect.prop('disabled', false);
}

function DoktorKutusunuKilitle() {
    var doktorSelect = $('#doktorSelect');
    doktorSelect.empty();
    doktorSelect.append('<option value="">-- Lütfen Önce Branş Seçiniz --</option>');
    doktorSelect.prop('disabled', true);
}

function SunucuHatasi() {
    alert("Doktor listesi çekilirken sunucu ile iletişim koptu.");
}
```

Çıktı: (yok — yan etki: kullanıcı branş seçtiği anda doktor `<select>` ögesi sunucudan gelen verilerle doldurulur ve aktif hale getirilir; bağlantı kopması durumunda alert gösterilir)

## 12.5 `success` ve `error` Callback'leri

### 12.5.1 `success` Ne Zaman Çalışır

- HTTP 200, 201, 204 gibi 2xx kodlu yanıt geldiğinde
- Yanıt gövdesi başarılı parse edildiğinde
- Parametre olarak sunucudan gelen veri (otomatik JSON parse edilir)

### 12.5.2 `error` Ne Zaman Çalışır

- HTTP 4xx (Bad Request, Not Found, Unauthorized)

- HTTP 5xx (Internal Server Error)
- Bağlantı kopması (ağ hatası, timeout)
- JSON parse hatası

Parametreler: `xhr` (XMLHttpRequest nesnesi), `status` ("error", "timeout"), `errorThrown` (hata metni).

## 12.6 GET vs POST AJAX

| Senaryo                        | HTTP metodu      | Neden                               |
|--------------------------------|------------------|-------------------------------------|
| Liste çekme, arama, filtreleme | GET              | Veri değiştirmiyor, cache'lenebilir |
| Yeni kayıt oluşturma           | POST             | Veri değiştiriyor, gövdede taşınır  |
| Mevcut kayıt güncelleme        | POST veya PUT    | Yan etki var                        |
| Silme                          | POST veya DELETE | Yan etki var, GET olmamalı          |

### 12.6.1 POST ile AJAX Örneği

```
// Views/Randevu/Ekle.cshtml - örnek POST çağırısı
$.ajax({
  url: '/Randevu/HizliKayit',
  type: 'POST',
  data: {
    HastaId: 5,
    DoktorId: 7,
    TarihSaat: '2025-06-15T10:30'
  },
  success: function (sonuc) {
    if (sonuc.basarili) {
      window.location.href = '/Randevu/Index';
    }
  }
});
```

## 12.7 Bölüm Özeti

- AJAX, sayfayı yenilemeden sunucudan veri çeker / sunucuya veri gönderir.
- `$.ajax()` ana metot; `url`, `type`, `data`, `success`, `error` temel parametreler.
- Controller'da `return Json(nesne)` ile JSON yanıt döndürülür; property adları camelCase'e çevrilir.
- `success` HTTP 2xx için, `error` 4xx/5xx ve ağ hataları için çağrılır.
- Bağımlı dropdown'lar (Branş → Doktor) AJAX'ın klasik kullanımudur.
- POST'a bağlı olmayan select'lerde `asp-for` kullanılmaz; sadece `id` ve `asp-items` yazılır.

## Bölüm 13 — CRUD İşlem Akışları

### 13.1 Genel Bakış

CRUD, dört temel veritabanı işleminin baş harfleridir:

- **C** — Create (oluştur)
- **R** — Read (oku)
- **U** — Update (güncelle)
- **D** — Delete (sil)

MVC'de her CRUD işleminin **bir veya iki action** ve bir view'ı vardır. Standart isimlendirme:

| İşlem        | Action adı           | URL                           | HTTP       |
|--------------|----------------------|-------------------------------|------------|
| Listele      | <code>Index</code>   | <code>/Hasta/Index</code>     | GET        |
| Detay göster | <code>Detay</code>   | <code>/Hasta/Detay/5</code>   | GET        |
| Yeni kayıt   | <code>Ekle</code>    | <code>/Hasta/Ekle</code>      | GET + POST |
| Güncelle     | <code>Duzenle</code> | <code>/Hasta/Duzenle/5</code> | GET + POST |
| Sil          | <code>Sil</code>     | <code>/Hasta/Sil/5</code>     | GET + POST |

### 13.2 LIST — `Index` Action ve View

#### 13.2.1 Controller

```
// Controllers/HastaController.cs
public IActionResult Index()
{
    var hastalar = _context.Hastalar.ToList();
    return View(hastalar);
}
```

#### 13.2.2 View

```
@* Views/Hasta/Index.cshtml *@
@model List<Hasta>

<div class="container mt-4">
    <div class="d-flex justify-content-between mb-3">
        <h4>Hasta Listesi</h4>
        <a asp-action="Ekle" class="btn btn-primary">+ Yeni Hasta</a>
    </div>

    <table class="table table-hover">
        <thead>
            <tr>
                <th>Ad Soyad</th>
                <th>TC</th>
                <th>İşlem</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var item in Model)
            {
                <tr>
                    <td>@item.Adi</td>
                    <td>@item.TcKimlik</td>
                    <td>
                        <a asp-action="Detay" asp-route-id="@item.Id" class="btn btn-sm btn-info">Detay</a>
                        <a asp-action="Duzenle" asp-route-id="@item.Id" class="btn btn-sm btn-warning">Düzenle</a>
                        <form asp-action="Sil" asp-route-id="@item.Id"
                            method="post" style="display:inline">
                            <button type="submit"
                                class="btn btn-sm btn-danger"
                                onclick="return confirm('Silinsin mi?')">
                                Sil
                            </button>
                        </form>
                    </td>
                </tr>
            }
        </tbody>
    </table>
</div>
```

Çıktı: (yok — yan etki: tarayıcıda tüm hastaları gösteren bir tablo render edilir; her satırda Detay, Düzenle, Sil butonları vardır)

## 13.3 CREATE — Ekle Action Çifti

### 13.3.1 Controller

```
// Controllers/HastaController.cs
public IActionResult Ekle()
{
    var vm = new HastaEkleVM
    {
        Hasta = new Hasta(),
        SehirListesi = _context.Sehirler.ToList()
    };
    return View(vm);
}

[HttpPost]
public IActionResult Ekle(HastaEkleVM vm)
{
    if (!ModelState.IsValid)
    {
        vm.SehirListesi = _context.Sehirler.ToList();
        return View(vm);
    }
    _context.Hastalar.Add(vm.Hasta);
    _context.SaveChanges();
    return RedirectToAction("Index");
}
```

### 13.3.2 View

```
@* Views/Hasta/Ekle.cshtml *@
@model HastaEkleVM

<div class="container mt-4">
    <div class="card shadow border-0">
        <div class="card-header bg-primary text-white">
            <h5>Yeni Hasta Ekle</h5>
        </div>
        <div class="card-body">
            <form asp-action="Ekle" method="post">

                <div class="mb-3">
                    <label asp-for="Hasta.Adi" class="form-label fw-bold">Ad Soyad</label>
                    <input asp-for="Hasta.Adi" class="form-control" required />
                    <span asp-validation-for="Hasta.Adi" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label asp-for="Hasta.SehirId" class="form-label fw-bold">Şehir</label>
                    <select asp-for="Hasta.SehirId"
                        asp-items="@((new SelectList(Model.SehirListesi, "Id", "Ad")))"
                        class="form-select" required>
                        <option value="">-- Şehir Seçiniz --</option>
                    </select>
                    <span asp-validation-for="Hasta.SehirId" class="text-danger"></span>
                </div>

                <a asp-action="Index" class="btn btn-secondary">İptal</a>
                <button type="submit" class="btn btn-primary">Kaydet</button>
            </form>
        </div>
    </div>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}
```

## 13.4 UPDATE — Düzenle Action Çifti

### 13.4.1 Controller

```
// Controllers/HastaController.cs
public IActionResult Düzenle(int id)
{
    var hasta = _context.Hastalar.Find(id);
    if (hasta == null) return NotFound();

    var vm = new HastaEkleVM
    {
        Hasta = hasta,
        SehirListesi = _context.Sehirler.ToList()
    };
    return View(vm);
}

[HttpPost]
public IActionResult Düzenle(HastaEkleVM vm)
{
    if (!ModelState.IsValid)
    {
        vm.SehirListesi = _context.Sehirler.ToList();
        return View(vm);
    }
    _context.Hastalar.Update(vm.Hasta);
    _context.SaveChanges();
    return RedirectToAction("Index");
}
```

### 13.4.2 View — Ekle'den Kritik Fark

Düzenleme view'ı, ekleme view'ına çok benzerdir; **iki kritik farkla**:

- Formun başına `<input type="hidden" asp-for="Hasta.Id" />` eklenir.
- Select'ler `asp-for` ile bağlı oldukları için mevcut değer otomatik `selected` gelir.

```
@* Views/Hasta/Duzenle.cshtml *@
@model HastaEkleVM

<div class="container mt-4">
    <div class="card border-0 shadow">
        <div class="card-header bg-warning text-dark">
            <h5>Hasta Düzenle</h5>
        </div>
        <div class="card-body">
            <form asp-action="Duzenle" method="post">

                @* ID'yi gizli taşı - kritik *@
                <input type="hidden" asp-for="Hasta.Id" />

                <div class="mb-3">
                    <label asp-for="Hasta.Adi" class="form-label fw-bold">Ad Soyad</label>
                    <input asp-for="Hasta.Adi" class="form-control" required />
                    <span asp-validation-for="Hasta.Adi" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label asp-for="Hasta.SehirId" class="form-label fw-bold">Şehir</label>
                    <select asp-for="Hasta.SehirId"
                        asp-items="@((new SelectList(Model.SehirListesi, "Id", "Ad")))"
                        class="form-select"></select>
                </div>

                <a asp-action="Index" class="btn btn-secondary">Vazgeç</a>
                <button type="submit" class="btn btn-warning text-dark">Güncelle</button>
            </form>
        </div>
    </div>
</div>
```

### 13.4.3 Hidden ID Önemi

Hidden alanı eklenmezse POST gövdesinde `Hasta.Id` gelmeyecektir; varsayılan `int` değeri `0` olur. `_context.Hastalar.Update(vm.Hasta)` çağrısında EF Core, `Id=0` olan kaydı yeni bir kayıt olarak değerlendirebilir veya kayıt bulamayıp hata fırlatabilir.

## 13.5 DETAIL — Detay Action

### 13.5.1 Controller

```
// Controllers/HastaController.cs
public IActionResult Detay(int id)
{
    var hasta = _context.Hastalar
        .Include(h => h.Sehir) // ilişkili tabloyu da yükle
        .FirstOrDefault(h => h.Id == id);

    if (hasta == null) return NotFound();
    return View(hasta);
}
```

### 13.5.2 View

```
@* Views/Hasta/Detay.cshtml *@
@model Hasta

<div class="container mt-4">
    <div class="card border-0 shadow">
        <div class="card-header bg-info text-white">
            <h5>Hasta Detayı</h5>
        </div>
        <div class="card-body">
            <dl class="row">
                <dt class="col-sm-3">Ad Soyad</dt>
                <dd class="col-sm-9">@Model.Adi</dd>

                <dt class="col-sm-3">TC Kimlik</dt>
                <dd class="col-sm-9">@Model.TcKimlik</dd>

                <dt class="col-sm-3">Şehir</dt>
                <dd class="col-sm-9">@Model.Sehir?.Ad</dd>
            </dl>

            <a asp-action="Index" class="btn btn-secondary">Geri</a>
            <a asp-action="Duzenle" asp-route-id="@Model.Id" class="btn btn-warning">Düzenle</a>
        </div>
    </div>
</div>
```

## 13.6 DELETE — Sil Action Çifti

### 13.6.1 Controller

```
// Controllers/HastaController.cs
[HttpPost]
public IActionResult Sil(int id)
{
    var hasta = _context.Hastalar.Find(id);
    if (hasta != null)
    {
        _context.Hastalar.Remove(hasta);
        _context.SaveChanges();
    }

    return RedirectToAction("Index");
}
```

### 13.6.2 İki Yaklaşım

#### Yaklaşım 1 — Index'te inline POST formu

Bu yaklaşımda ayrı bir "Sil" view'ı yoktur; silme isteği, Index'teki butona basıldığında doğrudan POST edilir (bkz. §13.2.2). `confirm()` ile JavaScript onayı alınır.

#### Yaklaşım 2 — Ayrı onay sayfası

Bu yaklaşımda iki action vardır: GET ile onay sayfasını gösteren, POST ile fiilen silen.

```
// GET – onay sayfası
public IActionResult Sil(int id)
{
    var hasta = _context.Hastalar.Find(id);
    if (hasta == null) return NotFound();
    return View(hasta);
}

// POST – fiili silme
[HttpPost, ActionName("Sil")]
public IActionResult SilOnayla(int id)
{
    var hasta = _context.Hastalar.Find(id);
    if (hasta != null)
    {
        _context.Hastalar.Remove(hasta);
        _context.SaveChanges();
    }
    return RedirectToAction("Index");
}
```

`[ActionName("Sil")]` özniteliği, metot adının `SilOnayla` olmasına rağmen URL'de `Sil` olarak görüldüğünden emin olur — iki action'ın `Sil` adıyla çakışmasını engellemek için kullanılır.

## 13.7 CRUD Akış Tablosu

| İşlem   | HTTP | Action(lar)              | Yönlendirme sonrası  |
|---------|------|--------------------------|----------------------|
| Liste   | GET  | <code>Index</code>       | Görüntülenir         |
| Detay   | GET  | <code>Detay(id)</code>   | Görüntülenir         |
| Ekle    | GET  | <code>Ekle()</code>      | Form gösterilir      |
| Ekle    | POST | <code>Ekle(vm)</code>    | Index'e redirect     |
| Düzenle | GET  | <code>Duzenle(id)</code> | Form mevcut veri ile |
| Düzenle | POST | <code>Duzenle(vm)</code> | Index'e redirect     |
| Sil     | POST | <code>Sil(id)</code>     | Index'e redirect     |

## 13.8 Bölüm Özeti

- Her CRUD işlemi standart action adlarıyla uygulanır.
- Ekle/Düzenle için GET (formu göster) + POST (kaydet) çifti vardır.
- Güncelleme formunda `<input type="hidden" asp-for="Id" />` zorunludur.
- Liste view'larında inline POST form'ları ile satır içi silme yapılabilir.
- Detay view'larında `Include()` ile ilişkili tablolar tek sorguda yüklenir.
- Başarılı POST sonrası daima `RedirectToAction` kullanılır (POST-Redirect-GET kalıbı).

# Bölüm 14 — Dependency Injection ve DbContext

## 14.1 Genel Bakış

ASP.NET Core, **Dependency Injection (DI) Container**'ı yerleşik olarak sunar. Bir sınıfın ihtiyaç duyduğu nesneler (servis, repository, veritabanı bağlantısı) DI tarafından otomatik olarak sağlanır. Bu yapı SOLID'in **D — Dependency Inversion** ilkesinin doğal uygulamasıdır (bkz. §4.6).

DI Container üç ana ömür (lifetime) sunar:

| Ömür      | Anlamı                                                |
|-----------|-------------------------------------------------------|
| Singleton | Uygulama boyunca tek örnek                            |
| Scoped    | Her HTTP isteği için bir örnek (DbContext varsayılan) |
| Transient | Her istendiğinde yeni örnek                           |

## 14.2 DbContext Sınıfı

### 14.2.1 Tanım

`DbContext`, Entity Framework Core'un veritabanı bağlantı ve sorgu yönetim sınıfıdır. Her tablo için bir `DbSet<T>` property'si tanımlanır.

### 14.2.2 Söz Dizimi

```
// Data/HastaneContext.cs
public class HastaneContext : DbContext
{
    public HastaneContext(DbContextOptions<HastaneContext> options)
        : base(options) { }

    // Her DbSet — bir veritabanı tablosu
    public DbSet<Hasta> Hastalar { get; set; }
    public DbSet<Doktor> Doktorlar { get; set; }
    public DbSet<Brans> Branslar { get; set; }
    public DbSet<Randevu> Randevular { get; set; }
}
```

| Property                         | SQL karşılığı                         |
|----------------------------------|---------------------------------------|
| <code>_context.Hastalar</code>   | <code>SELECT * FROM Hastalar</code>   |
| <code>_context.Doktorlar</code>  | <code>SELECT * FROM Doktorlar</code>  |
| <code>_context.Branslar</code>   | <code>SELECT * FROM Branslar</code>   |
| <code>_context.Randevular</code> | <code>SELECT * FROM Randevular</code> |

## 14.3 Program.cs — DI Kayıt

### 14.3.1 DbContext'in DI Container'a Kaydı

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// 1) DbContext'i kaydet
builder.Services.AddDbContext<HastaneContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

// 2) MVC desteği
builder.Services.AddControllersWithViews();

var app = builder.Build();

// HTTP pipeline
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

### 14.3.2 Connection String — appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "YOUR_CONNECTION_STRING"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

Hassas bilgi notu: gerçek connection string sızdırılmamalıdır. Üretim ortamında User Secrets veya environment değişkeni kullanılır.

## 14.4 Constructor Injection

### 14.4.1 Söz Dizimi

```
// Controllers/BransController.cs
public class BransController : Controller
{
    // 1) private readonly alan — sadece bu sınıf kullanır, değişmez
    private readonly HastaneContext _context;

    // 2) Constructor — ASP.NET Core context'i otomatik verir
    public BransController(HastaneContext context)
    {
        _context = context;
    }

    // Tüm action'lar _context üzerinden veritabanına ulaşır
    public IActionResult Index()
    {
        var liste = _context.Branslar.ToList();
        return View(liste);
    }
}
```

#### 14.4.2 Her Satırın Görevi

| Satır                                                       | Görevi                                                                                                           |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>private readonly HastaneContext _context</code>       | Saklama alanı; <code>private</code> dış erişimi kapatır, <code>readonly</code> constructor dışı atamayı engeller |
| <code>public BransController(HastaneContext context)</code> | ASP.NET Core bunu çağırır; parametreyi DI Container'dan alır                                                     |
| <code>_context = context</code>                             | Gelen context'i alana atar                                                                                       |

#### 14.4.3 DI Çözümleme Akışı

```
sequenceDiagram
    participant T as Tarayıcı
    participant K as ASP.NET Core
    participant DI as DI Container
    participant C as BransController

    T->>K: GET /Brans/Index
    K->>DI: BransController istenir
    DI->>DI: Constructor parametresi: HastaneContext
    DI->>DI: HastaneContext kayıtlı mı? — Evet
    DI->>DI: HastaneContext örneği oluştur
    DI->>C: new BransController(context)
    C->>C: _context = context
    K->>C: Action çağırısı
    C-->>T: View
```

### 14.5 EF Core CRUD İşlemleri

#### 14.5.1 READ (Okuma)

```
// Controllers/BransController.cs - Örnek metotlar

// Tüm kayıtlar
var liste = _context.Branslar.ToList();

// ID'ye göre tek kayıt
var brans = _context.Branslar.Find(id);           // PK ile arama

// Koşullu filtreleme
var aktifler = _context.Branslar
    .Where(b => b.Aktif == true)
    .ToList();

// İlk eşleşeni getir
var ilk = _context.Branslar
    .FirstOrDefault(b => b.BransAdi == "Kardiyoloji");

// İlişkili tabloyla birlikte
var doktorlar = _context.Doktorlar
    .Include(d => d.Brans)                       // JOIN benzeri
    .ToList();

// LINQ projeksiyon
var ozet = _context.Doktorlar
    .Select(d => new { d.Id, d.AdSoyad })
    .ToList();
```

### 14.5.2 CREATE (Ekleme)

```
var yeni = new Brans
{
    BransAdi = "Nöroloji",
    Aktif    = true
};

_context.Branslar.Add(yeni);           // bellekte
_context.SaveChanges();               // veritabanına yaz
// Üretilen SQL: INSERT INTO Branslar (BransAdi, Aktif) VALUES ('Nöroloji', 1)
```

### 14.5.3 UPDATE (Güncelleme)

```
var brans = _context.Branslar.Find(id);
if (brans != null)
{
    brans.BransAdi = "Genel Cerrahi";
    _context.Branslar.Update(brans);
    _context.SaveChanges();
}
// Üretilen SQL: UPDATE Branslar SET BransAdi='Genel Cerrahi' WHERE Id=5
```

### 14.5.4 DELETE (Silme)

```
var brans = _context.Branslar.Find(id);
if (brans != null)
{
    _context.Branslar.Remove(brans);
    _context.SaveChanges();
}
// Üretilen SQL: DELETE FROM Branslar WHERE Id=5
```

## 14.6 `SaveChanges()` Davranışı

EF Core, bellekteki değişiklikleri **takip eder** (change tracking) ama veritabanına otomatik yazmaz. `SaveChanges()` çağrılana kadar:

- `Add()`, `Update()`, `Remove()` → bellekte
- `SaveChanges()` → tüm bekleyen değişiklikler tek transaction'da veritabanına gönderilir

### 14.6.1 Toplu İşlem Örneği

```
_context.Branslar.Add(new Brans { BransAdi = "Üroloji" });
_context.Branslar.Add(new Brans { BransAdi = "Dermatoloji" });

var silinecek = _context.Branslar.Find(99);
if (silinecek != null)
    _context.Branslar.Remove(silinecek);

_context.SaveChanges(); // 3 işlem tek seferde, transaction içinde
```

Eğer bu üç işlemten biri başarısız olursa **hiçbiri uygulanmaz** — atomicity sağlanır.

## 14.7 Tam Controller Örneği

```
// Controllers/BransController.cs
public class BransController : Controller
{
    private readonly HastaneContext _context;

    public BransController(HastaneContext context)
    {
        _context = context;
    }

    // — LIST —
    public IActionResult Index()
    {
        var liste = _context.Branslar.ToList();
        return View(liste);
    }

    // — CREATE GET —
    public IActionResult Ekle()
    {
        return View();
    }

    // — CREATE POST —
    [HttpPost]
    public IActionResult Ekle(Brans brans)
    {
        if (!ModelState.IsValid) return View(brans);

        _context.Branslar.Add(brans);
        _context.SaveChanges();
        return RedirectToAction("Index");
    }

    // — EDIT GET —
    public IActionResult Duzenle(int id)
    {
        var brans = _context.Branslar.Find(id);
        if (brans == null) return NotFound();
        return View(brans);
    }

    // — EDIT POST —
```

```

[HttpPost]
public IActionResult Duzenle(Brans brans)
{
    if (!ModelState.IsValid) return View(brans);

    _context.Branslar.Update(brans);
    _context.SaveChanges();
    return RedirectToAction("Index");
}

// — DELETE —————
[HttpPost]
public IActionResult Sil(int id)
{
    var brans = _context.Branslar.Find(id);
    if (brans != null)
    {
        _context.Branslar.Remove(brans);
        _context.SaveChanges();
    }
    return RedirectToAction("Index");
}
}

```

## 14.8 DI vs Manuel Nesne Yaratma

| Manuel ( new ) ile                           | DI ile                                           |
|----------------------------------------------|--------------------------------------------------|
| <code>_context = new HastaneContext()</code> | DI Container örneği sağlar                       |
| Connection string her yerde tekrar yazılır   | Tek yerde ( <code>Program.cs</code> ) tanımlanır |
| Bağlantı kapanması manuel yönetilir          | Request bitince otomatik dispose                 |
| Test için sahte (mock) context kullanılamaz  | Interface enjekte edilerek mock geçilir          |
| SOLID — DIP ihlali                           | SOLID uyumlu                                     |

## 14.9 Yapı Özeti

| Bileşen                             | Yer                                 | Görevi                              |
|-------------------------------------|-------------------------------------|-------------------------------------|
| Connection string                   | <code>appsettings.json</code>       | Veritabanı adresi                   |
| <code>HastaneContext</code>         | <code>Data/HastaneContext.cs</code> | Tabloları tanımlayan EF Core sınıfı |
| <code>AddDbContext</code>           | <code>Program.cs</code>             | Context'i DI Container'a kaydeder   |
| Constructor parametresi             | Her Controller                      | DI Container'dan context'i alır     |
| <code>_context.Tablo.İşlem()</code> | Action metotları                    | Veritabanı erişimi                  |
| <code>SaveChanges()</code>          | Add/Update/Remove sonrası           | Değişiklikleri kalıcılaştırır       |

## 14.10 Bölüm Özeti

- DI Container, controller constructor'ına gerekli nesneleri otomatik enjekte eder.
- `DbContext` EF Core'un veritabanı bağlantı sınıfıdır; her tablo bir `DbSet<T>` ile temsil edilir.
- `Program.cs` içinde `AddDbContext` ile DI'a kayıt yapılır; connection string `appsettings.json`'dan okunur.
- Controller'larda `private readonly HastaneContext _context` standart kalıptır.
- EF Core değişiklikleri takip eder; `SaveChanges()` ile tek transaction'da uygular.
- `Find(id)` PK ile, `Where(...)` koşulla, `Include(...)` ilişkili tablo dahil sorgular.



# Bölüm 15 — HastaneMVC Proje Dokümantasyonu

## 15.1 Proje Tanıtımı

HastaneMVC, ASP.NET Core MVC üzerine inşa edilmiş, **hasta-doktor-randevu** yönetimi yapan örnek bir uygulamadır. Bu bölüm önceki bölümlerde anlatılan tüm kavramların (Tag Helper, AJAX, DI, EF Core, ViewModel, validation) bir araya gelişini gösterir.

Temel iş kuralları:

- Bir hasta birden fazla randevu alabilir (1—N)
- Bir doktor birden fazla randevuya sahip olabilir (1—N)
- Bir branşta birden fazla doktor vardır (1—N)
- Randevu oluştururken **önce branş, sonra doktor** seçilir; doktor listesi branşa göre AJAX ile yüklenir

## 15.2 Proje Yapısı

```
HastaneMVC/
├── Controllers/
│   ├── HomeController.cs
│   ├── HastaController.cs
│   ├── BransController.cs
│   ├── DoktorController.cs
│   └── RandevuController.cs
├── Data/
│   └── HastaneContext.cs
├── Models/
│   ├── Hasta.cs
│   ├── Doktor.cs
│   ├── Brans.cs
│   └── Randevu.cs
├── ViewModels/
│   └── RandevuEkleVM.cs
├── Views/
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   └── _ValidationScriptsPartial.cshtml
│   ├── Randevu/
│   │   ├── Index.cshtml
│   │   ├── Ekle.cshtml
│   │   └── Guncelle.cshtml
│   ├── Brans/
│   │   └── ...
│   ├── _ViewStart.cshtml
│   └── _ViewImports.cshtml
├── wwwroot/
│   ├── lib/
│   │   ├── jquery/
│   │   ├── bootstrap/
│   │   └── jquery-validation-unobtrusive/
│   └── css/
├── appsettings.json
└── Program.cs
```

Not: Bu ağaç, sohbet boyunca tanımlanan veya ima edilen dosyaları içerir. `HomeController`, `DoktorController` gibi bazı dosyalar standart MVC iskeletinin parçası olarak varsayılmıştır; sohbette doğrudan paylaşılmadılar.

## 15.3 Veri Katmanı

### 15.3.1 Entity'ler

```
// Models/Hasta.cs
public class Hasta
{
    public int    Id        { get; set; }
    public string AdSoyad  { get; set; }
    public string TcKimlik { get; set; }

    // Navigation
    public ICollection<Randevu> Randevular { get; set; } = new List<Randevu>();
}
```

```
// Models/Brans.cs
public class Brans
{
    public int    Id        { get; set; }
    public string BransAdi { get; set; }
    public bool   Aktif     { get; set; } = true;

    // Navigation – bir branşa çok doktor
    public ICollection<Doktor> Doktorlar { get; set; } = new List<Doktor>();
}
```

```
// Models/Doktor.cs
public class Doktor
{
    public int    Id        { get; set; }
    public string AdSoyad { get; set; }

    // FK – bir doktor bir branşa bağlı
    public int    BransId { get; set; }
    public Brans Brans    { get; set; }

    // Navigation – bir doktorun çok randevusu
    public ICollection<Randevu> Randevular { get; set; } = new List<Randevu>();
}
```

```
// Models/Randevu.cs
public class Randevu
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Hasta seçimi zorunludur")]
    public int HastaId { get; set; }
    public Hasta Hasta { get; set; }

    [Required(ErrorMessage = "Doktor seçimi zorunludur")]
    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }

    [Required(ErrorMessage = "Tarih zorunludur")]
    public DateTime TarihSaat { get; set; }

    public string DoktorNotu { get; set; }
}
```

### 15.3.2 İlişki Şeması

```

erDiagram
    HASTA ||--o{ RANDEVU : "alır"
    DOKTOR ||--o{ RANDEVU : "hizmet verir"
    BRANS ||--o{ DOKTOR : "içerir"

    HASTA {
        int Id PK
        string AdSoyad
        string TcKimlik
    }
    DOKTOR {
        int Id PK
        int BransId FK
        string AdSoyad
    }
    BRANS {
        int Id PK
        string BransAdi
    }
    RANDEVU {
        int Id PK
        int HastaId FK
        int DoktorId FK
        datetime TarihSaat
        string DoktorNotu
    }

```

### 15.3.3 DbContext

```

// Data/HastaneContext.cs
public class HastaneContext : DbContext
{
    public HastaneContext(DbContextOptions<HastaneContext> options)
        : base(options) { }

    public DbSet<Hasta> Hastalar { get; set; }
    public DbSet<Doktor> Doktorlar { get; set; }
    public DbSet<Brans> Branslar { get; set; }
    public DbSet<Randevu> Randevular { get; set; }
}

```

### 15.3.4 Program.cs

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<HastaneContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

### 15.3.5 Yapılandırma

```
// appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection": "YOUR_CONNECTION_STRING"
  }
}
```

## 15.4 ViewModel Katmanı

### 15.4.1 RandevuEkleVM

```
// ViewModels/RandevuEkleVM.cs
public class RandevuEkleVM
{
    // Form verisi — kaydedilecek randevu
    public Randevu YeniRandevu { get; set; } = new();

    // Açılır liste verileri
    public List<Hasta> HastaListesi { get; set; } = new();
    public List<Brans> BransListesi { get; set; } = new();
    public List<Doktor> DoktorListesi { get; set; } = new();
}
```

ViewModel'in amacı: tek bir view'da hem form için kullanılacak **Randevu** nesnesini hem de açılır listeler için ham verileri (**Hasta**, **Brans**, **Doktor** listeleri) taşımak.

## 15.5 Özellik — Randevu Ekleme

### 15.5.1 Amaç

Kullanıcı yeni bir randevu kaydı oluşturur. Form üç açılır liste ve bir tarih alanından oluşur:

1. **Hasta** — POST'a bağlı, model ile bind edilir
2. **Brans** — POST'a bağlı değil, sadece doktor listesini AJAX ile filtrelemek için kullanılır
3. **Doktor** — POST'a bağlı, başlangıçta boş ve **disabled**; brans seçilince AJAX ile dolar
4. **Tarih ve Saat** — **datetime-local** input

## 15.5.2 Akış

```
sequenceDiagram
    participant K as Kullanıcı
    participant T as Tarayıcı (jQuery)
    participant C as RandevuController
    participant DB as HastaneContext

    K->>T: /Randevu/Ekle açar
    T->>C: GET Ekle()
    C->>DB: Hastalar, Branşlar, (boş) Doktorlar
    DB-->>C: Listeler
    C-->>T: View(vm)
    T-->>K: Form (Hasta + Branş dolu, Doktor boş)

    K->>T: Branş seçer
    T->>T: change olayı
    T->>C: GET DoktorlariGetir?bransId=X
    C->>DB: Doktorlar.Where(BransId == X)
    DB-->>C: Doktor listesi
    C-->>T: JSON
    T->>T: Doktor select doldurulur
    T-->>K: Doktor aktif

    K->>T: Hasta, Doktor, Tarih seçer
    K->>T: Submit
    T->>C: POST Ekle(vm)
    C->>C: ModelState.IsValid?
    C->>DB: Randevular.Add(vm.YeniRandevu)
    DB->>DB: SaveChanges()
    C-->>T: Redirect /Randevu/Index
```

## 15.5.3 Controller

```
// Controllers/RandevuController.cs
public class RandevuController : Controller
{
    private readonly HastaneContext _context;

    public RandevuController(HastaneContext context)
    {
        _context = context;
    }

    // GET — formu göster
    public IActionResult Ekle()
    {
        var vm = new RandevuEkleVM
        {
            YeniRandevu = new Randevu(),
            HastaListesi = _context.Hastalar.ToList(),
            BransListesi = _context.Branslar.ToList(),
            DoktorListesi = new List<Doktor>() // başta boş
        };
        return View(vm);
    }

    // POST — kaydet
    [HttpPost]
    public IActionResult Ekle(RandevuEkleVM vm)
    {
        if (!ModelState.IsValid)
        {
            // Hata varsa listeleri yeniden yükle
            vm.HastaListesi = _context.Hastalar.ToList();
            vm.BransListesi = _context.Branslar.ToList();
            vm.DoktorListesi = new List<Doktor>();
            return View(vm);
        }

        _context.Randevular.Add(vm.YeniRandevu);
        _context.SaveChanges();
        return RedirectToAction("Index");
    }

    // AJAX — branşa göre doktorları döndür
    public IActionResult DoktorlariGetir(int bransId)
    {
        var doktorlar = _context.Doktorlar
            .Where(d => d.BransId == bransId)
            .Select(d => new { d.Id, d.AdSoyad })
            .ToList();

        return Json(doktorlar);
    }
}

```

## 15.5.4 View

```
@* Views/Randevu/Ekle.cshtml *@
@using HastaneMVC.ViewModels
@model RandevuEkleVM

@{
    ViewData["Title"] = "Yeni Randevu Oluştur";
}

<div class="container mt-5">
    <div class="row justify-content-center">

```

```

<div class="col-md-8">
    <div class="card shadow-lg border-0">
        <div class="card-header bg-primary text-white py-3">
            <h5 class="mb-0">
                <i class="bi bi-calendar-plus-fill"></i>
                Yeni Randevu Kayıt Formu
            </h5>
        </div>
        <div class="card-body p-4">
            <form asp-action="Ekle" method="post">

                @* HASTA *@
                <div class="mb-4">
                    <label asp-for="YeniRandevu.HastaId" class="form-label fw-bold">
                        Randevu Alacak Hasta
                    </label>
                    <div class="input-group">
                        <span class="input-group-text bg-light">
                            <i class="bi bi-person"></i>
                        </span>
                        <select asp-for="YeniRandevu.HastaId"
                            asp-items="@ (new SelectList(Model.HastaListesi, "Id", "AdSoyad"))"
                            class="form-select form-select-lg" required>
                            <option value="">-- Lütfen Hasta Seçiniz --</option>
                        </select>
                    </div>
                    <span asp-validation-for="YeniRandevu.HastaId" class="text-danger"></span>
                </div>

                @* BRANŞ — POST'a bağlı değil *@
                <div class="mb-4">
                    <label class="form-label fw-bold">Brans</label>
                    <div class="input-group">
                        <span class="input-group-text bg-light">
                            <i class="bi bi-diagram-3"></i>
                        </span>
                        <select id="bransSelect"
                            asp-items="@ (new SelectList(Model.BransListesi, "Id", "BransAdi"))"
                            class="form-select form-select-lg" required>
                            <option value="">-- Önce Branş Seçiniz --</option>
                        </select>
                    </div>
                </div>

                @* DOKTOR — başta disabled, AJAX ile dolar *@
                <div class="mb-4">
                    <label asp-for="YeniRandevu.DoktorId" class="form-label fw-bold">
                        Gidilecek Doktor
                    </label>
                    <div class="input-group">
                        <span class="input-group-text bg-light">
                            <i class="bi bi-file-medical"></i>
                        </span>
                        <select id="doktorSelect"
                            asp-for="YeniRandevu.DoktorId"
                            class="form-select form-select-lg" required disabled>
                            <option value="">-- Lütfen Önce Branş Seçiniz --</option>
                        </select>
                    </div>
                    <div class="form-text text-muted">
                        <i class="bi bi-info-circle"></i>
                        Doktor listesi seçilen branşa göre otomatik yüklenecektir.
                    </div>
                    <span asp-validation-for="YeniRandevu.DoktorId" class="text-danger"></span>
                </div>

                @* TARİH-SAAT *@

```

```

        <div class="mb-4">
            <label asp-for="YeniRandevu.TarihSaat" class="form-label fw-bold">
                Randevu Tarihi ve Saati
            </label>
            <div class="input-group">
                <span class="input-group-text bg-light">
                    <i class="bi bi-clock"></i>
                </span>
                <input asp-for="YeniRandevu.TarihSaat"
                    type="datetime-local"
                    class="form-control form-control-lg" required />
            </div>
            <span asp-validation-for="YeniRandevu.TarihSaat" class="text-danger"></span>
        </div>

        <hr class="my-4" />

        <div class="d-grid gap-2 d-md-flex justify-content-md-end">
            <a asp-action="Index" class="btn btn-outline-secondary px-4">
                <i class="bi bi-arrow-left"></i> Listeye Dön
            </a>
            <button type="submit" class="btn btn-primary px-5 fw-bold">
                <i class="bi bi-calendar-check"></i> Randevuyu Onayla
            </button>
        </div>

    </form>
</div>
</div>
</div>
</div>
</div>

```

```

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }

    <script>
        $(document).ready(function () {
            $('#bransSelect').change(BransDegisti);
        });

        function BransDegisti() {
            var secilenBransId = $('#bransSelect').val();
            if (secilenBransId !== "") {
                AjaxIleDoktorlariGetir(secilenBransId);
            } else {
                DoktorKutusunuKilitle();
            }
        }

        function AjaxIleDoktorlariGetir(bransId) {
            $.ajax({
                url:      '/Randevu/DoktorlariGetir',
                type:     'GET',
                data:     { bransId: bransId },
                success:  DoktorKutusunuDoldur,
                error:    SunucuHatasi
            });
        }

        function DoktorKutusunuDoldur(gelenVeri) {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Doktor Seçiniz --</option>');
            for (var i = 0; i < gelenVeri.length; i++) {
                var d = gelenVeri[i];
                doktorSelect.append(

```

```

        '<option value="' + d.id + '"' + d.adSoyad + '</option>'
    );
}
doktorSelect.prop('disabled', false);
}

function DoktorKutusunuKilitle() {
    var doktorSelect = $('#doktorSelect');
    doktorSelect.empty();
    doktorSelect.append('<option value="">-- Lütfen Önce Branş Seçiniz --</option>');
    doktorSelect.prop('disabled', true);
}

function SunucuHatasi() {
    alert("Doktor listesi çekilirken sunucu ile iletişim koptu. Lütfen sayfayı yenileyin.");
}
</script>
}

```

### 15.5.5 Beklenen Sonuçlar

| Senaryo                                       | Sonuç                                                                  |
|-----------------------------------------------|------------------------------------------------------------------------|
| Tüm alanlar dolu, geçerli                     | Kayıt oluşturulur, Index'e yönlendirilir                               |
| Hasta seçilmeden submit                       | Form yeniden gösterilir, "Hasta seçimi zorunludur" hatası alan altında |
| Doktor seçilmeden submit                      | Aynı şekilde "Doktor seçimi zorunludur" hatası                         |
| Branş seçildi ancak doktor yüklenmeden submit | Doktor alanı boş kalır, validation hatası verir                        |
| AJAX sırasında bağlantı kopar                 | <code>SunucuHatasi()</code> çağrılır, alert gösterilir                 |

### 15.5.6 Tasarım Notları

- **Branş POST'a gönderilmez:** `asp-for` yazılmadığı için `name` üretilmez. Yalnızca `id="bransSelect"` ile jQuery'ye trigger olarak hizmet eder.
- **Doktor başlangıçta disabled:** Branş seçilmeden doktor seçimi mantıksızdır.
- **JSON property adları:** Controller'da `Select(d => new { d.Id, d.AdSoyad })` yazılır; tarayıcıya `id` ve `adSoyad` olarak gelir (camelCase).
- **Hata sonrası listelerin yeniden yüklenmesi:** POST'ta validation hatası olursa `vm.HastaListesi` ve `vm.BransListesi` yeniden doldurulmaz ise dropdown'lar boş kalır; bu yüzden controller'da yeniden çekme zorunludur.

## 15.6 Özellik — Randevu Güncelleme

### 15.6.1 Amaç

Mevcut bir randevu kaydını düzenler. Ekle formundan üç temel farkı vardır:

1. **Hidden ID alanı** — ID'yi taşımak için
2. **Form alanları mevcut değerlerle doldurulur gelir** — `asp-for` sayesinde otomatik
3. **Triyaj / Doktor Notu alanı eklenir** — randevu öncesi/sırasında not düşmek için

### 15.6.2 Controller

```
// Controllers/RandevuController.cs
public IActionResult Guncelle(int id)
{
    var randevu = _context.Randevular
        .Include(r => r.Doktor)
        .FirstOrDefault(r => r.Id == id);

    if (randevu == null) return NotFound();

    var vm = new RandevuEkleVM
    {
        YeniRandevu = randevu,
        HastaListesi = _context.Hastalar.ToList(),
        BransListesi = _context.Branslar.ToList(),
        DoktorListesi = _context.Doktorlar
            .Where(d => d.BransId == randevu.Doktor.BransId)
            .ToList()
    };
    return View(vm);
}

[HttpPost]
public IActionResult Guncelle(RandevuEkleVM vm)
{
    if (!ModelState.IsValid)
    {
        vm.HastaListesi = _context.Hastalar.ToList();
        vm.BransListesi = _context.Branslar.ToList();
        vm.DoktorListesi = _context.Doktorlar.ToList();
        return View(vm);
    }

    _context.Randevular.Update(vm.YeniRandevu);
    _context.SaveChanges();
    return RedirectToAction("Index");
}
}
```

### 15.6.3 View

```
@* Views/Randevu/Guncelle.cshtml *@
@using HastaneMVC.ViewModels
@model RandevuEkleVM

@{
    ViewData["Title"] = "Randevu Güncelle / Not Ekle";
}

<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-8">
            <div class="card shadow border-0">
                <div class="card-header bg-warning text-dark py-3">
                    <h5 class="mb-0">
                        <i class="bi bi-pencil-square"></i>
                        Randevu Bilgilerini Düzenle
                    </h5>
                </div>
                <div class="card-body p-4">
                    <form asp-action="Guncelle" method="post">

                        @* ID gizli alan — kritik *@
                        <input type="hidden" asp-for="YeniRandevu.Id" />

                        <div class="row">
                            @* HASTA *@
```

```

        <div class="col-md-4 mb-4">
            <label asp-for="YeniRandevu.HastaId" class="form-label fw-bold">Hasta</label>
            <select asp-for="YeniRandevu.HastaId"
                asp-items="@ (new SelectList(Model.HastaListesi, "Id", "AdSoyad"))"
                class="form-select" required>
            </select>
        </div>

        @* BRANŞ *@
        <div class="col-md-4 mb-4">
            <label class="form-label fw-bold">Branş</label>
            <select id="bransSelect"
                asp-items="@ (new SelectList(Model.BransListesi, "Id", "BransAdi"))"
                class="form-select" required>
            </select>
        </div>

        @* DOKTOR *@
        <div class="col-md-4 mb-4">
            <label asp-for="YeniRandevu.DoktorId" class="form-label fw-bold">Doktor</label>
            <select id="doktorSelect"
                asp-for="YeniRandevu.DoktorId"
                asp-items="@ (new SelectList(Model.DoktorListesi, "Id", "AdSoyad"))"
                class="form-select" required>
            </select>
        </div>
    </div>

    @* TARİH *@
    <div class="mb-4">
        <label asp-for="YeniRandevu.TarihSaat" class="form-label fw-bold">
            Randevu Tarihi ve Saati
        </label>
        <input asp-for="YeniRandevu.TarihSaat"
            type="datetime-local"
            class="form-control" required />
    </div>

    @* TRIYAJ NOTU *@
    <div class="mb-4">
        <label asp-for="YeniRandevu.DoktorNotu" class="form-label fw-bold text-primary">
            Triyaj
        </label>
        <textarea asp-for="YeniRandevu.DoktorNotu"
            class="form-control"
            rows="4"
            placeholder="Trijaj notu..."></textarea>
    </div>

    <hr class="my-4" />

    <div class="d-grid gap-2 d-md-flex justify-content-md-end">
        <a asp-action="Index" class="btn btn-outline-secondary px-4">İptal</a>
        <button type="submit" class="btn btn-warning px-5 fw-bold text-dark">
            <i class="bi bi-save"></i> Değişiklikleri Kaydet
        </button>
    </div>
</form>
</div>
</div>
</div>
</div>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}

```

```

<script>
    $(document).ready(function () {
        $('#bransSelect').change(BransDegisti);
    });

    function BransDegisti() {
        var bransId = $('#bransSelect').val();
        if (bransId !== "") {
            $.ajax({
                url: '/Randevu/DoktorlariGetir',
                type: 'GET',
                data: { bransId: bransId },
                success: function (gelen) {
                    var sel = $('#doktorSelect');
                    sel.empty();
                    sel.append('<option value="">-- Doktor Seçiniz --</option>');
                    for (var i = 0; i < gelen.length; i++) {
                        sel.append('<option value="' + gelen[i].id + '">' + gelen[i].adSoyad + '</option>');
                    }
                }
            });
        }
    }
}
</script>
}

```

#### 15.6.4 Beklenen Sonuçlar

| Senaryo                       | Sonuç                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------|
| Geçerli ID ile sayfa açma     | Form mevcut değerler ile dolu gelir                                                             |
| Var olmayan ID ile sayfa açma | HTTP 404 Not Found                                                                              |
| Bir alan değişip submit       | Kayıt güncellenir, Index'e yönlendirilir                                                        |
| Hidden ID kaldırılırsa        | <code>YeniRandevu.Id=0</code> olur, EF Core update yerine insert deneyebilir veya hata fırlatır |
| Tüm alanlar boş submit        | Validation hataları gösterilir                                                                  |

#### 15.6.5 Tasarım Notları

- **Hidden ID kritiktir:** Form gönderiminde `Id` taşınmazsa EF Core'un `Update()` çağrısı yanlış kayıt üzerinde işlem yapar veya hata verir.
- **Doktor listesi sayfa yüklendiğinde önceden dolu gelir:** Controller'da, mevcut doktorun branşına ait diğer doktorlar VM'in `DoktorListesi` 'ne yüklenir. `asp-for="YeniRandevu.DoktorId"` mevcut doktoru otomatik `selected` yapar.
- **Branş select'i değer taşıyor:** `asp-for` yoksa POST'a gitmez; sadece doktor listesini yeniden yüklemek için kullanılır.

### 15.7 Bölüm Özeti

- HastaneMVC, 4 ana entity (Hasta, Doktor, Brans, Randevu) ile 1—N ilişkilerden oluşur.
- `RandevuEkleVM` ViewModel hem yeni randevu hem güncelleme formlarında kullanılır.
- Branş → Doktor AJAX çağrısı `DoktorlariGetir(int bransId)` action'ı üzerinden JSON döner.
- Ekle ve Güncelle formları aynı view yapısını paylaşır; tek fark hidden ID alanı ve doktor listesinin önden yüklenmesidir.
- Tüm controller'lar constructor injection ile `HastaneContext` alır.

## Ek A — Tag Helper Hızlı Referansı

| HTML Etiketi     | Tag Helper                                | Üretilen Attribute(lar)           |
|------------------|-------------------------------------------|-----------------------------------|
| <form>           | asp-action , asp-controller               | action                            |
| <a>              | asp-action , asp-controller , asp-route-* | href                              |
| <label>          | asp-for                                   | for                               |
| <input> (text)   | asp-for                                   | id , name , type , value          |
| <input> (hidden) | asp-for + type="hidden"                   | id , name , value                 |
| <textarea>       | asp-for                                   | id , name , içerik                |
| <select> (bind)  | asp-for , asp-items                       | id , name , option'lar , selected |
| <select> (AJAX)  | sadece asp-items + id                     | option'lar (name yok)             |
| <span>           | asp-validation-for                        | hata mesajı                       |
| <div>            | asp-validation-summary                    | tüm hata listesi                  |

### Üç altın kural:

- asp-for → label · input · textarea · select etiketlerinde
- asp-action → form · a etiketlerinde
- asp-items → sadece select etiketinde

## Ek B — Terim Sözlüğü

| Terim (TR / EN)                         | Açıklama                                                                                               |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Sınıf / Class</b>                    | Nesnelerin yapısını ve davranışlarını tanımlayan şablon                                                |
| <b>Nesne / Object</b>                   | Bir sınıftan üretilen somut örnek                                                                      |
| <b>Property / Özellik</b>               | Nesnenin durum bilgisini taşıyan üye                                                                   |
| <b>Field / Alan</b>                     | Sınıf içi değişken; genelde <code>private</code>                                                       |
| <b>Method / Metot</b>                   | Sınıfın davranışını tanımlayan üye                                                                     |
| <b>Constructor / Kurucu metot</b>       | Nesne oluşturulurken otomatik çalışan özel metot                                                       |
| <b>Encapsulation / Kapsülleme</b>       | İç veriyi gizleyip kontrollü erişim sağlama                                                            |
| <b>Inheritance / Kalıtım</b>            | Bir sınıfın başka sınıfın üyelerini miras alması                                                       |
| <b>Polymorphism / Çok biçimlilik</b>    | Üst sınıf referansı üzerinden alt sınıf davranışının çalışması                                         |
| <b>Abstraction / Soyutlama</b>          | Karmaşıklığı gizleyip sade arayüz sunma                                                                |
| <b>Interface / Arayüz</b>               | Üye imzalarını içeren saf sözleşme                                                                     |
| <b>Abstract class / Soyut sınıf</b>     | Doğrudan örneklenemeyen, kısmi şablon sunan sınıf                                                      |
| <b>SOLID</b>                            | Beş yazılım tasarım ilkesi: SRP, OCP, LSP, ISP, DIP                                                    |
| <b>Primary Key (PK)</b>                 | Tablodaki her kaydı benzersiz tanımlayan sütun                                                         |
| <b>Foreign Key (FK)</b>                 | Başka tablonun PK'sına işaret eden sütun                                                               |
| <b>Composite Key</b>                    | İki veya daha fazla sütunun birlikte PK oluşturduğu yapı                                               |
| <b>Junction Table / Ara tablo</b>       | N—M ilişkileri çözmek için iki FK içeren tablo                                                         |
| <b>Cascade Delete / Basamaklı silme</b> | Ana kayıt silindiğinde bağlı alt kayıtların otomatik silinmesi                                         |
| <b>ORM</b>                              | Object-Relational Mapping; nesneleri tablolarla eşleştiren teknoloji                                   |
| <b>DbContext</b>                        | EF Core'un veritabanı bağlantı ve sorgu sınıfı                                                         |
| <b>DbSet&lt;T&gt;</b>                   | DbContext içinde bir tabloyu temsil eden generic property                                              |
| <b>LINQ</b>                             | C#'ın koleksiyon sorgu API'si                                                                          |
| <b>Razor</b>                            | ASP.NET Core'un view şablon dili                                                                       |
| <b>Tag Helper</b>                       | Razor'da HTML etiketlerine <code>asp-</code> özellikleriyle eklenen yardımcı                           |
| <b>Model Binding</b>                    | Form verisinin C# nesnelere otomatik eşlenmesi                                                         |
| <b>ViewModel (VM)</b>                   | View'a özel hazırlanmış, birden çok veri parçasını taşıyan sınıf                                       |
| <b>Dependency Injection (DI)</b>        | Bağımlılıkların sınıfın dışından enjekte edilmesi                                                      |
| <b>Constructor Injection</b>            | Bağımlılığın constructor parametresi olarak alınması                                                   |
| <b>AJAX</b>                             | Asenkron JavaScript ve XML; sayfa yenilemeden istek                                                    |
| <b>JSON</b>                             | JavaScript Object Notation; AJAX yanıtlarının standart formatı                                         |
| <b>POST-Redirect-GET</b>                | Başarılı POST sonrası redirect yapma kalıbı                                                            |
| <b>Action</b>                           | Controller'da HTTP isteğini karşılayan public metot                                                    |
| <b>Action Result</b>                    | Action'ın döndürdüğü yanıt tipi ( <code>View</code> , <code>Redirect</code> , <code>Json</code> , ...) |